

11.1 How to read this chapter

This chapter describes the USB controller which is present on all LPC176x/5x devices except the LPC1767. On some LPC176x/5x family devices, the USB controller can also be configured for Host or OTG operation.

11.2 Basic configuration

The USB controller is configured using the following registers:

1. Power: In the PCONP register ([Table 46](#)), set bit PCUSB.
Remark: On reset, the USB block is disabled (PCUSB = 0).
2. Clock: The USB block can be used with a dedicated USB PLL (PLL1) to obtain the USB clock or with the Main PLL (PLL0). See [Section 4.6.1](#).
3. Pins: Select USB pins and their modes in PINSEL0 to PINSEL5 and PINMODE0 to PINMODE5 ([Section 8.5](#)).
4. Wake-up: Activity on the USB bus port can wake up the microcontroller from Power-down mode, see [Section 4.8.8](#).
5. Interrupts: Interrupts are enabled in the NVIC using the appropriate Interrupt Set Enable register.
6. Initialization: see [Section 11.13](#).

11.3 Introduction

The Universal Serial Bus (USB) is a four-wire bus that supports communication between a host and one or more (up to 127) peripherals. The host controller allocates the USB bandwidth to attached devices through a token-based protocol. The bus supports hot plugging and dynamic configuration of the devices. All transactions are initiated by the host controller.

The host schedules transactions in 1 ms frames. Each frame contains a Start-Of-Frame (SOF) marker and transactions that transfer data to or from device endpoints. Each device can have a maximum of 16 logical or 32 physical endpoints. There are four types of transfers defined for the endpoints. Control transfers are used to configure the device. Interrupt transfers are used for periodic data transfer. Bulk transfers are used when the rate of transfer is not critical. Isochronous transfers have guaranteed delivery time but no error correction.

For more information on the Universal Serial Bus, see the USB Implementers Forum website.

The USB device controller on the LPC176x/5x enables full-speed (12 Mb/s) data exchange with a USB host controller.

Table 185. USB related acronyms, abbreviations, and definitions used in this chapter

Acronym/abbreviation	Description
AHB	Advanced High-performance bus
ATLE	Auto Transfer Length Extraction
ATX	Analog Transceiver
DD	DMA Descriptor
DDP	DMA Description Pointer
DMA	Direct Memory Access
EOP	End-Of-Packet
EP	Endpoint
EP_RAM	Endpoint RAM
FS	Full Speed
LED	Light Emitting Diode
LS	Low Speed
MPS	Maximum Packet Size
NAK	Negative Acknowledge
PLL	Phase Locked Loop
RAM	Random Access Memory
SOF	Start-Of-Frame
SIE	Serial Interface Engine
SRAM	Synchronous RAM
UDCA	USB Device Communication Area
USB	Universal Serial Bus

11.4 Features

- Fully compliant with the USB 2.0 specification (full speed).
- Supports 32 physical (16 logical) endpoints.
- Supports Control, Bulk, Interrupt and Isochronous endpoints.
- Scalable realization of endpoints at run time.
- Endpoint maximum packet size selection (up to USB maximum specification) by software at run time.
- Supports SoftConnect and GoodLink features.
- Supports DMA transfers on all non-control endpoints.
- Allows dynamic switching between CPU controlled and DMA modes.
- Double buffer implementation for Bulk and Isochronous endpoints.

11.5 Fixed endpoint configuration

[Table 186](#) shows the supported endpoint configurations. Endpoints are realized and configured at run time using the Endpoint realization registers, documented in [Section 11.10.4 “Endpoint realization registers”](#).

Table 186. Fixed endpoint configuration

Logical endpoint	Physical endpoint	Endpoint type	Direction	Packet size (bytes)	Double buffer
0	0	Control	Out	8, 16, 32, 64	No
0	1	Control	In	8, 16, 32, 64	No
1	2	Interrupt	Out	1 to 64	No
1	3	Interrupt	In	1 to 64	No
2	4	Bulk	Out	8, 16, 32, 64	Yes
2	5	Bulk	In	8, 16, 32, 64	Yes
3	6	Isochronous	Out	1 to 1023	Yes
3	7	Isochronous	In	1 to 1023	Yes
4	8	Interrupt	Out	1 to 64	No
4	9	Interrupt	In	1 to 64	No
5	10	Bulk	Out	8, 16, 32, 64	Yes
5	11	Bulk	In	8, 16, 32, 64	Yes
6	12	Isochronous	Out	1 to 1023	Yes
6	13	Isochronous	In	1 to 1023	Yes
7	14	Interrupt	Out	1 to 64	No
7	15	Interrupt	In	1 to 64	No
8	16	Bulk	Out	8, 16, 32, 64	Yes
8	17	Bulk	In	8, 16, 32, 64	Yes
9	18	Isochronous	Out	1 to 1023	Yes
9	19	Isochronous	In	1 to 1023	Yes
10	20	Interrupt	Out	1 to 64	No
10	21	Interrupt	In	1 to 64	No
11	22	Bulk	Out	8, 16, 32, 64	Yes
11	23	Bulk	In	8, 16, 32, 64	Yes
12	24	Isochronous	Out	1 to 1023	Yes
12	25	Isochronous	In	1 to 1023	Yes
13	26	Interrupt	Out	1 to 64	No
13	27	Interrupt	In	1 to 64	No
14	28	Bulk	Out	8, 16, 32, 64	Yes
14	29	Bulk	In	8, 16, 32, 64	Yes
15	30	Bulk	Out	8, 16, 32, 64	Yes
15	31	Bulk	In	8, 16, 32, 64	Yes

11.6 Functional description

The architecture of the USB device controller is shown below in [Figure 28](#).

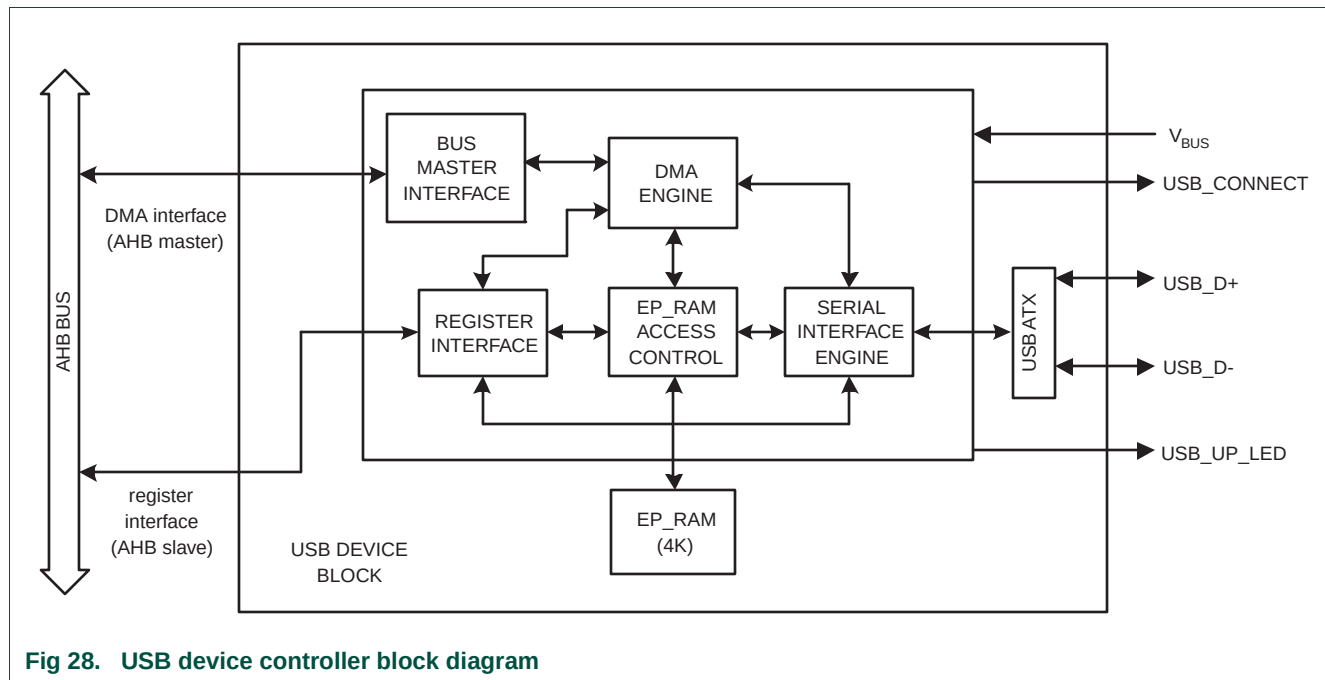


Fig 28. USB device controller block diagram

11.6.1 Analog transceiver

The USB Device Controller has a built-in analog transceiver (ATX). The USB ATX sends/receives the bi-directional D+ and D- signals of the USB bus.

11.6.2 Serial Interface Engine (SIE)

The SIE implements the full USB protocol layer. It is completely hardwired for speed and needs no firmware intervention. It handles transfer of data between the endpoint buffers in EP_RAM and the USB bus. The functions of this block include: synchronization pattern recognition, parallel/serial conversion, bit stuffing/de-stuffing, CRC checking/generation, PID verification/generation, address recognition, and handshake evaluation/generation.

11.6.3 Endpoint RAM (EP_RAM)

Each endpoint buffer is implemented as an SRAM based FIFO. The SRAM dedicated for this purpose is called the EP_RAM. Each realized endpoint has a reserved space in the EP_RAM. The total EP_RAM space required depends on the number of realized endpoints, the maximum packet size of the endpoint, and whether the endpoint supports double buffering.

11.6.4 EP_RAM access control

The EP_RAM Access Control logic handles transfer of data from/to the EP_RAM and the three sources that can access it: the CPU (via the Register Interface), the SIE, and the DMA Engine.

11.6.5 DMA engine and bus master interface

When enabled for an endpoint, the DMA Engine transfers data between RAM on the AHB bus and the endpoint's buffer in EP_RAM. A single DMA channel is shared between all endpoints. When transferring data, the DMA Engine functions as a master on the AHB bus through the bus master interface.

11.6.6 Register interface

The Register Interface allows the CPU to control the operation of the USB Device Controller. It also provides a way to write transmit data to the controller and read receive data from the controller.

11.6.7 SoftConnect

The connection to the USB is accomplished by bringing D+ (for a full-speed device) HIGH through a 1.5 kOhm pull-up resistor. The SoftConnect feature can be used to allow software to finish its initialization sequence before deciding to establish connection to the USB. Re-initialization of the USB bus connection can also be performed without having to unplug the cable.

To use the SoftConnect feature, the CONNECT signal should control an external switch that connects the 1.5 kOhm resistor between D+ and +3.3V. Software can then control the CONNECT signal by writing to the CON bit using the SIE Set Device Status command.

11.6.8 GoodLink

Good USB connection indication is provided through GoodLink technology. When the device is successfully enumerated and configured, the LED indicator will be permanently ON. During suspend, the LED will be OFF.

This feature provides a user-friendly indicator on the status of the USB device. It is a useful field diagnostics tool to isolate faulty equipment.

To use the GoodLink feature the UP_LED signal should control an LED. The UP_LED signal is controlled using the SIE Configure Device command.

11.7 Operational overview

Transactions on the USB bus transfer data between device endpoints and the host. The direction of a transaction is defined with respect to the host. OUT transactions transfer data from the host to the device. IN transactions transfer data from the device to the host. All transactions are initiated by the host controller.

For an OUT transaction, the USB ATX receives the bi-directional D+ and D- signals of the USB bus. The Serial Interface Engine (SIE) receives the serial data from the ATX and converts it into a parallel data stream. The parallel data is written to the corresponding endpoint buffer in the EP_RAM.

For IN transactions, the SIE reads the parallel data from the endpoint buffer in EP_RAM, converts it into serial data, and transmits it onto the USB bus using the USB ATX.

Once data has been received or sent, the endpoint buffer can be read or written. How this is accomplished depends on the endpoint's type and operating mode. The two operating modes for each endpoint are Slave (CPU-controlled) mode, and DMA mode.

In Slave mode, the CPU transfers data between RAM and the endpoint buffer using the Register Interface. See [Section 11.14 “Slave mode operation”](#) for a detailed description of this mode.

In DMA mode, the DMA transfers data between RAM and the endpoint buffer. See [Section 11.15 “DMA operation”](#) for a detailed description of this mode.

11.8 Pin description

Table 187. USB external interface

Name	Direction	Description
V _{BUS}	I	V _{BUS} status input. When this function is not enabled via its corresponding PINSEL register, it is driven HIGH internally.
USB_CONNECT	O	SoftConnect control signal.
USB_UP_LED	O	GoodLink LED control signal.
USB_D+	I/O	Positive differential data.
USB_D-	I/O	Negative differential data.

11.9 Clocking and power management

This section describes the clocking and power management features of the USB Device Controller.

11.9.1 Power requirements

The USB protocol insists on power management by the device. This becomes very critical if the device draws power from the bus (bus-powered device). The following constraints should be met by a bus-powered device:

1. A device in the non-configured state should draw a maximum of 100 mA from the bus.
2. A configured device can draw only up to what is specified in the Max Power field of the configuration descriptor. The maximum value is 500 mA.
3. A suspended device can draw a maximum of 2.5 mA.

11.9.2 Clocks

The USB device controller clocks are shown in [Table 188](#)

Table 188. USB device controller clock sources

Clock source	Description
AHB master clock	Clock for the AHB master bus interface and DMA
AHB slave clock	Clock for the AHB slave interface
usbclk	48 MHz clock from the dedicated USB PLL (PLL1) or the Main PLL (PLL0), used to recover the 12 MHz clock from the USB bus

11.9.3 Power management support

To help conserve power, the USB device controller automatically disables the AHB master clock and usbclk when not in use.

When the USB Device Controller goes into the suspend state (bus is idle for 3 ms), the usbclk input to the device controller is automatically disabled, helping to conserve power. However, if software wishes to access the device controller registers, usbclk must be active. To allow access to the device controller registers while in the suspend state, the USBClkCtrl and USBClkSt registers are provided.

When software wishes to access the device controller registers, it should first ensure usbclk is enabled by setting DEV_CLK_EN in the USBClkCtrl register, and then poll the corresponding DEV_CLK_ON bit in USBClkSt until set. Once set, usbclk will remain enabled until DEV_CLK_EN is cleared by software.

When a DMA transfer occurs, the device controller automatically turns on the AHB master clock. Once asserted, it remains active for a minimum of 2 ms (2 frames), to help ensure that DMA throughput is not affected by turning off the AHB master clock. 2 ms after the last DMA access, the AHB master clock is automatically disabled to help conserve power. If desired, software also has the capability of forcing this clock to remain enabled using the USBClkCtrl register.

Note that the AHB slave clock is always enabled as long as the PCUSB bit of PCONP is set. When the device controller is not in use, all of the device controller clocks may be disabled by clearing PCUSB.

The USB_NEED_CLK signal is used to facilitate going into and waking up from chip Power-down mode. USB_NEED_CLK is asserted if any of the bits of the USBClkSt register are asserted.

After entering the suspend state with DEV_CLK_EN and AHB_CLK_EN cleared, the DEV_CLK_ON and AHB_CLK_ON will be cleared when the corresponding clock turns off. When both bits are zero, USB_NEED_CLK will be low, indicating that the chip can be put into Power-down mode by writing to the PCON register. The status of USB_NEED_CLK can be read from the USBIntSt register.

Any bus activity in the suspend state will cause the USB_NEED_CLK signal to be asserted. When the chip is in Power-down mode and the USB interrupt is enabled, the assertion of USB_NEED_CLK causes the chip to wake up from Power-down mode.

11.9.4 Remote wake-up

The USB device controller supports software initiated remote wake-up. Remote wake-up involves resume signaling on the USB bus initiated from the device. This is done by clearing the SUS bit in the SIE Set Device Status register. Before writing into the register, all the clocks to the device controller have to be enabled using the USBClkCtrl register.

11.10 Register description

[Table 189](#) shows the USB Device Controller registers directly accessible by the CPU. The Serial Interface Engine (SIE) has other registers that are indirectly accessible via the SIE command registers. See [Section 11.12 “Serial interface engine command description”](#) for more info.

Table 189. USB device register map

Name	Description	Access	Reset value ^[1]	Address
Clock control registers				
USBClkCtrl	USB Clock Control	R/W	0	0x5000 CFF4
USBClkSt	USB Clock Status	RO	0	0x5000 CFF8
Device interrupt registers				
USBIntSt	USB Interrupt Status	R/W	0x8000 0000	0x400F C1C0
USBDevIntSt	USB Device Interrupt Status	RO	0x10	0x5000 C200
USBDevIntEn	USB Device Interrupt Enable	R/W	0	0x5000 C204
USBDevIntClr	USB Device Interrupt Clear	WO	0	0x5000 C208
USBDevIntSet	USB Device Interrupt Set	WO	0	0x5000 C20C
USBDevIntPri	USB Device Interrupt Priority	WO	0	0x5000 C22C
Endpoint interrupt registers				
USBEpIntSt	USB Endpoint Interrupt Status	RO	0	0x5000 C230
USBEpIntEn	USB Endpoint Interrupt Enable	R/W	0	0x5000 C234
USBEpIntClr	USB Endpoint Interrupt Clear	WO	0	0x5000 C238
USBEpIntSet	USB Endpoint Interrupt Set	WO	0	0x5000 C23C
USBEpIntPri	USB Endpoint Priority	WO ^[2]	0	0x5000 C240
Endpoint realization registers				
USBReEp	USB Realize Endpoint	R/W	0x3	0x5000 C244
USBEpIn	USB Endpoint Index	WO ^[2]	0	0x5000 C248
USBMaxPSize	USB MaxPacketSize	R/W	0x8	0x5000 C24C
USB transfer registers				
USBRxData	USB Receive Data	RO	0	0x5000 C218
USBRxPLen	USB Receive Packet Length	RO	0	0x5000 C220
USBTxData	USB Transmit Data	WO ^[2]	0	0x5000 C21C
USBTxPLen	USB Transmit Packet Length	WO ^[2]	0	0x5000 C224
USBCtrl	USB Control	R/W	0	0x5000 C228
SIE Command registers				
USBCmdCode	USB Command Code	WO ^[2]	0	0x5000 C210
USBCmdData	USB Command Data	RO	0	0x5000 C214

Table 189. USB device register map ...continued

Name	Description	Access	Reset value ^[1]	Address
DMA registers				
USBDMARSt	USB DMA Request Status	RO	0	0x5000 C250
USBDMARClr	USB DMA Request Clear	WO ^[2]	0	0x5000 C254
USBDMARSet	USB DMA Request Set	WO ^[2]	0	0x5000 C258
USBUDCAH	USB UDCA Head	R/W	0	0x5000 C280
USBEPDMASSt	USB Endpoint DMA Status	RO	0	0x5000 C284
USBEPDMAEn	USB Endpoint DMA Enable	WO ^[2]	0	0x5000 C288
USBEPDMADis	USB Endpoint DMA Disable	WO ^[2]	0	0x5000 C28C
USBDMAIntSt	USB DMA Interrupt Status	RO	0	0x5000 C290
USBDMAIntEn	USB DMA Interrupt Enable	R/W	0	0x5000 C294
USBEoTIntSt	USB End of Transfer Interrupt Status	RO	0	0x5000 C2A0
USBEoTIntClr	USB End of Transfer Interrupt Clear	WO ^[2]	0	0x5000 C2A4
USBEoTIntSet	USB End of Transfer Interrupt Set	WO ^[2]	0	0x5000 C2A8
USBNDDRIntSt	USB New DD Request Interrupt Status	RO	0	0x5000 C2AC
USBNDDRIntClr	USB New DD Request Interrupt Clear	WO ^[2]	0	0x5000 C2B0
USBNDDRIntSet	USB New DD Request Interrupt Set	WO ^[2]	0	0x5000 C2B4
USBSysErrIntSt	USB System Error Interrupt Status	RO	0	0x5000 C2B8
USBSysErrIntClr	USB System Error Interrupt Clear	WO ^[2]	0	0x5000 C2BC
USBSysErrIntSet	USB System Error Interrupt Set	WO ^[2]	0	0x5000 C2C0

[1] Reset value reflects the data stored in used bits only. It does not include reserved bits content.

[2] Reading WO register will return an invalid value.

11.10.1 Clock control registers

11.10.1.1 USB Clock Control register (USBClkCtrl - 0x5000 CFF4)

This register controls the clocking of the USB Device Controller. Whenever software wants to access the device controller registers, both DEV_CLK_EN and AHB_CLK_EN must be set.

The software does not have to repeat this exercise for every register access, provided that the corresponding USBClkCtrl bits are already set. Note that this register is functional only when the PCUSB bit of PCONP is set; when PCUSB is cleared, all clocks to the device controller are disabled irrespective of the contents of this register. USBClkCtrl is a read/write register.

Table 190. USBClkCtrl register (USBClkCtrl - address 0x5000 CFF4) bit description

Bit	Symbol	Description	Reset value
0	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA
1	DEV_CLK_EN	Device clock enable. Enables the usbclk input to the device controller	0
2	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

Table 190. USBClkCtrl register (USBClkCtrl - address 0x5000 CFF4) bit description ...continued

Bit	Symbol	Description	Reset value
3	-	Reserved. User software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA
4	AHB_CLK_EN	AHB clock enable	0
31:5	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

11.10.1.2 USB Clock Status register (USBClkSt - 0x5000 CFF8)

This register holds the clock availability status. The bits of this register are ORed together to form the USB_NEED_CLK signal. When enabling a clock via USBClkCtrl, software should poll the corresponding bit in USBClkSt. If it is set, then software can go ahead with the register access. Software does not have to repeat this exercise for every access, provided that the USBClkCtrl bits are not disturbed. USBClkSt is a read-only register.

Table 191. USB Clock Status register (USBClkSt - address 0x5000 CFF8) bit description

Bit	Symbol	Description	Reset value
0	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA
1	DEV_CLK_ON	Device clock on. The usbclk input to the device controller is active.	0
3:2	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA
4	AHB_CLK_ON	AHB clock on.	0
31:5	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

11.10.2 Device interrupt registers

11.10.2.1 USB Interrupt Status register (USBIntSt - 0x5000 C1C0)

The USB Device Controller has three interrupt lines. This register allows software to determine their status with a single read operation. All three interrupt lines are ORed together to a single channel of the vectored interrupt controller. This register also contains the USB_NEED_CLK status and EN_USB_INTS control bits. USBIntSt is a read/write register.

Table 192. USB Interrupt Status register (USBIntSt - address 0x5000 C1C0) bit description

Bit	Symbol	Description	Reset value
0	USB_INT_REQ_LP	Low priority interrupt line status. This bit is read-only.	0
1	USB_INT_REQ_HP	High priority interrupt line status. This bit is read-only.	0
2	USB_INT_REQ_DMA	DMA interrupt line status. This bit is read-only.	0
7:3	-	These bits are reserved in a device-only configuration. User software should not write ones to reserved bits. The value read from a reserved bit is not defined. See Table 258 for OTG configuration.	NA

Table 192. USB Interrupt Status register (USBIntSt - address 0x5000 C1C0) bit description ...continued

Bit	Symbol	Description	Reset value
8	USB_NEED_CLK	USB need clock indicator. This bit is set to 1 when USB activity or a change of state on the USB data pins is detected, and it indicates that a PLL supplied clock of 48 MHz is needed. Once USB_NEED_CLK becomes one, it resets to zero 5 ms after the last packet has been received/sent, or 2 ms after the Suspend Change (SUS_CH) interrupt has occurred. A change of this bit from 0 to 1 can wake up the microcontroller if activity on the USB bus is selected to wake up the part from the Power-down mode (see Section 4.8.8 "Wake-up from Reduced Power Modes" for details). Also see Section 4.5.9 "PLL0 and Power-down mode" and Section 4.8.9 "Power Control for Peripherals register (PCONP - 0x400F C0C4)" for considerations about the PLL and invoking the Power-down mode. This bit is read-only.	1
30:9	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA
31	EN_USB_INTS	Enable all USB interrupts. When this bit is cleared, the Vectored Interrupt Controller does not see the ORed output of the USB interrupt lines.	1

11.10.2.2 USB Device Interrupt Status register (USBDevIntSt - 0x5000 C200)

The USBDevIntSt register holds the status of each interrupt. A 0 indicates no interrupt and 1 indicates the presence of the interrupt. USBDevIntSt is a read-only register.

Table 193. USB Device Interrupt Status register (USBDevIntSt - address 0x5000 C200) bit allocation

Reset value: 0x0000 0000

Bit	31	30	29	28	27	26	25	24
Symbol	-	-	-	-	-	-	-	-
Bit	23	22	21	20	19	18	17	16
Symbol	-	-	-	-	-	-	-	-
Bit	15	14	13	12	11	10	9	8
Symbol	-	-	-	-	-	-	ERR_INT	EP_RLZED
Bit	7	6	5	4	3	2	1	0
Symbol	TxENDPKT	Rx ENDPKT	CDFULL	CCEMPTY	DEV_STAT	EP_SLOW	EP_FAST	FRAME

Table 194. USB Device Interrupt Status register (USBDevIntSt - address 0x5000 C200) bit description

Bit	Symbol	Description	Reset value
0	FRAME	The frame interrupt occurs every 1 ms. This is used in isochronous packet transfers.	0
1	EP_FAST	Fast endpoint interrupt. If an Endpoint Interrupt Priority register (USBEPIntPri) bit is set, the corresponding endpoint interrupt will be routed to this bit.	0
2	EP_SLOW	Slow endpoints interrupt. If an Endpoint Interrupt Priority Register (USBEPIntPri) bit is not set, the corresponding endpoint interrupt will be routed to this bit.	0
3	DEV_STAT	Set when USB Bus reset, USB suspend change or Connect change event occurs. Refer to Section 11.12.6 "Set Device Status (Command: 0xFE, Data: write 1 byte)" on page 257 .	0
4	CCEMPTY	The command code register (USBCmdCode) is empty (New command can be written).	1
5	CDFULL	Command data register (USBCmdData) is full (Data can be read now).	0
6	RxENDPKT	The current packet in the endpoint buffer is transferred to the CPU.	0
7	TxENDPKT	The number of data bytes transferred to the endpoint buffer equals the number of bytes programmed in the TxPacket length register (USBTxPLen).	0

Table 194. USB Device Interrupt Status register (USBDevIntSt - address 0x5000 C200) bit description ...continued

Bit	Symbol	Description	Reset value
8	EP_RLZED	Endpoints realized. Set when Realize Endpoint register (USBReEp) or MaxPacketSize register (USBMaxPSize) is updated and the corresponding operation is completed.	0
9	ERR_INT	Error Interrupt. Any bus error interrupt from the USB device. Refer to Section 11.12.9 "Read Error Status (Command: 0xFB, Data: read 1 byte)" on page 259	0
31:10	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

11.10.2.3 USB Device Interrupt Enable register (USBDevIntEn - 0x5000 C204)

Writing a one to a bit in this register enables the corresponding bit in USBDevIntSt to generate an interrupt on one of the interrupt lines when set. By default, the interrupt is routed to the USB_INT_REQ_LP interrupt line. Optionally, either the EP_FAST or FRAME interrupt may be routed to the USB_INT_REQ_HP interrupt line by changing the value of USBDevIntPri. USBDevIntEn is a read/write register.

Table 195. USB Device Interrupt Enable register (USBDevIntEn - address 0x5000 C204) bit allocation

Reset value: 0x0000 0000

Bit	31	30	29	28	27	26	25	24
Symbol	-	-	-	-	-	-	-	-
Bit	23	22	21	20	19	18	17	16
Symbol	-	-	-	-	-	-	-	-
Bit	15	14	13	12	11	10	9	8
Symbol	-	-	-	-	-	-	ERR_INT	EP_RLZED
Bit	7	6	5	4	3	2	1	0
Symbol	TxENDPKT	Rx ENDPKT	CDFULL	CCEMPTY	DEV_STAT	EP_SLOW	EP_FAST	FRAME

Table 196. USB Device Interrupt Enable register (USBDevIntEn - address 0x5000 C204) bit description

Bit	Symbol	Value	Description	Reset value
31:0	See USBDevIntEn bit allocation table above	0	No interrupt is generated.	0
		1	An interrupt will be generated when the corresponding bit in the Device Interrupt Status (USBDevIntSt) register (Table 193) is set. By default, the interrupt is routed to the USB_INT_REQ_LP interrupt line. Optionally, either the EP_FAST or FRAME interrupt may be routed to the USB_INT_REQ_HP interrupt line by changing the value of USBDevIntPri.	

11.10.2.4 USB Device Interrupt Clear register (USBDevIntClr - 0x5000 C208)

Writing one to a bit in this register clears the corresponding bit in USBDevIntSt. Writing a zero has no effect.

Remark: Before clearing the EP_SLOW or EP_FAST interrupt bits, the corresponding endpoint interrupts in USBEpIntSt should be cleared.

USBDevIntClr is a write-only register.

Table 197. USB Device Interrupt Clear register (USBDevIntClr - address 0x5000 C208) bit allocation

Reset value: 0x0000 0000

Bit	31	30	29	28	27	26	25	24
Symbol	-	-	-	-	-	-	-	-
Bit	23	22	21	20	19	18	17	16
Symbol	-	-	-	-	-	-	-	-
Bit	15	14	13	12	11	10	9	8
Symbol	-	-	-	-	-	-	ERR_INT	EP_RLZED
Bit	7	6	5	4	3	2	1	0
Symbol	TxENDPKT	Rx ENDPKT	CDFULL	CCEMPTY	DEV_STAT	EP_SLOW	EP_FAST	FRAME

Table 198. USB Device Interrupt Clear register (USBDevIntClr - address 0x5000 C208) bit description

Bit	Symbol	Value	Description	Reset value
31:0	See USBDevIntClr bit allocation table above	0	No effect.	0
		1	The corresponding bit in USBDevIntSt (Section 11.10.2.2) is cleared.	

11.10.2.5 USB Device Interrupt Set register (USBDevIntSet - 0x5000 C20C)

Writing one to a bit in this register sets the corresponding bit in the USBDevIntSt. Writing a zero has no effect

USBDevIntSet is a write-only register.

Table 199. USB Device Interrupt Set register (USBDevIntSet - address 0x5000 C20C) bit allocation

Reset value: 0x0000 0000

Bit	31	30	29	28	27	26	25	24
Symbol	-	-	-	-	-	-	-	-
Bit	23	22	21	20	19	18	17	16
Symbol	-	-	-	-	-	-	-	-
Bit	15	14	13	12	11	10	9	8
Symbol	-	-	-	-	-	-	ERR_INT	EP_RLZED
Bit	7	6	5	4	3	2	1	0
Symbol	TxENDPKT	Rx ENDPKT	CDFULL	CCEMPTY	DEV_STAT	EP_SLOW	EP_FAST	FRAME

Table 200. USB Device Interrupt Set register (USBDevIntSet - address 0x5000 C20C) bit description

Bit	Symbol	Value	Description	Reset value
31:0	See USBDevIntSet bit allocation table above	0	No effect.	0
		1	The corresponding bit in USBDevIntSt (Section 11.10.2.2) is set.	

11.10.2.6 USB Device Interrupt Priority register (USBDevIntPri - 0x5000 C22C)

Writing one to a bit in this register causes the corresponding interrupt to be routed to the USB_INT_REQ_HP interrupt line. Writing zero causes the interrupt to be routed to the USB_INT_REQ_LP interrupt line. Either the EP_FAST or FRAME interrupt can be routed to USB_INT_REQ_HP, but not both. If the software attempts to set both bits to one, no interrupt will be routed to USB_INT_REQ_HP. USBDevIntPri is a write-only register.

Table 201. USB Device Interrupt Priority register (USBDevIntPri - address 0x5000 C22C) bit description

Bit	Symbol	Value	Description	Reset value
0	FRAME	0	FRAME interrupt is routed to USB_INT_REQ_LP.	0
		1	FRAME interrupt is routed to USB_INT_REQ_HP.	
1	EP_FAST	0	EP_FAST interrupt is routed to USB_INT_REQ_LP.	0
		1	EP_FAST interrupt is routed to USB_INT_REQ_HP.	
31:2	-		Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

11.10.3 Endpoint interrupt registers

The registers in this group facilitate handling of endpoint interrupts. Endpoint interrupts are used in Slave mode operation.

11.10.3.1 USB Endpoint Interrupt Status register (USBEPIntSt - 0x5000 C230)

Each physical non-isochronous endpoint is represented by a bit in this register to indicate that it has generated an interrupt. All non-isochronous OUT endpoints generate an interrupt when they receive a packet without an error. All non-isochronous IN endpoints generate an interrupt when a packet is successfully transmitted, or when a NAK handshake is sent on the bus and the interrupt on NAK feature is enabled (see [Section 11.12.3 “Set Mode \(Command: 0xF3, Data: write 1 byte\)” on page 256](#)). A bit set to one in this register causes either the EP_FAST or EP_SLOW bit of USBDevIntSt to be set depending on the value of the corresponding bit of USBEPDevIntPri. USBEPIntSt is a read-only register.

Note that for Isochronous endpoints, handling of packet data is done when the FRAME interrupt occurs.

Table 202. USB Endpoint Interrupt Status register (USBEPIntSt - address 0x5000 C230) bit allocation

Reset value: 0x0000 0000

Bit	31	30	29	28	27	26	25	24
Symbol	EP15TX	EP15RX	EP14TX	EP14RX	EP13TX	EP13RX	EP12TX	EP12RX
Bit	23	22	21	20	19	18	17	16
Symbol	EP11TX	EP11RX	EP10TX	EP10RX	EP9TX	EP9RX	EP8TX	EP8RX
Bit	15	14	13	12	11	10	9	8
Symbol	EP7TX	EP7RX	EP6TX	EP6RX	EP5TX	EP5RX	EP4TX	EP4RX
Bit	7	6	5	4	3	2	1	0
Symbol	EP3TX	EP3RX	EP2TX	EP2RX	EP1TX	EP1RX	EP0TX	EP0RX

Table 203. USB Endpoint Interrupt Status register (USBEPIntSt - address 0x5000 C230) bit description

Bit	Symbol	Description	Reset value
0	EP0RX	Endpoint 0, Data Received Interrupt bit.	0
1	EP0TX	Endpoint 0, Data Transmitted Interrupt bit or sent a NAK.	0
2	EP1RX	Endpoint 1, Data Received Interrupt bit.	0
3	EP1TX	Endpoint 1, Data Transmitted Interrupt bit or sent a NAK.	0
4	EP2RX	Endpoint 2, Data Received Interrupt bit.	0
5	EP2TX	Endpoint 2, Data Transmitted Interrupt bit or sent a NAK.	0
6	EP3RX	Endpoint 3, Isochronous endpoint.	NA
7	EP3TX	Endpoint 3, Isochronous endpoint.	NA
8	EP4RX	Endpoint 4, Data Received Interrupt bit.	0
9	EP4TX	Endpoint 4, Data Transmitted Interrupt bit or sent a NAK.	0
10	EP5RX	Endpoint 5, Data Received Interrupt bit.	0
11	EP5TX	Endpoint 5, Data Transmitted Interrupt bit or sent a NAK.	0
12	EP6RX	Endpoint 6, Isochronous endpoint.	NA
13	EP6TX	Endpoint 6, Isochronous endpoint.	NA
14	EP7RX	Endpoint 7, Data Received Interrupt bit.	0
15	EP7TX	Endpoint 7, Data Transmitted Interrupt bit or sent a NAK.	0
16	EP8RX	Endpoint 8, Data Received Interrupt bit.	0
17	EP8TX	Endpoint 8, Data Transmitted Interrupt bit or sent a NAK.	0
18	EP9RX	Endpoint 9, Isochronous endpoint.	NA
19	EP9TX	Endpoint 9, Isochronous endpoint.	NA
20	EP10RX	Endpoint 10, Data Received Interrupt bit.	0
21	EP10TX	Endpoint 10, Data Transmitted Interrupt bit or sent a NAK.	0
22	EP11RX	Endpoint 11, Data Received Interrupt bit.	0
23	EP11TX	Endpoint 11, Data Transmitted Interrupt bit or sent a NAK.	0
24	EP12RX	Endpoint 12, Isochronous endpoint.	NA
25	EP12TX	Endpoint 12, Isochronous endpoint.	NA
26	EP13RX	Endpoint 13, Data Received Interrupt bit.	0
27	EP13TX	Endpoint 13, Data Transmitted Interrupt bit or sent a NAK.	0
28	EP14RX	Endpoint 14, Data Received Interrupt bit.	0
29	EP14TX	Endpoint 14, Data Transmitted Interrupt bit or sent a NAK.	0
30	EP15RX	Endpoint 15, Data Received Interrupt bit.	0
31	EP15TX	Endpoint 15, Data Transmitted Interrupt bit or sent a NAK.	0

11.10.3.2 USB Endpoint Interrupt Enable register (USBEPIntEn - 0x5000 C234)

Setting a bit to 1 in this register causes the corresponding bit in USBEPIntSt to be set when an interrupt occurs for the associated endpoint. Setting a bit to 0 causes the corresponding bit in USBDMARSt to be set when an interrupt occurs for the associated endpoint. USBEPIntEn is a read/write register.

Table 204. USB Endpoint Interrupt Enable register (USBEPIntEn - address 0x5000 C234) bit allocation

Reset value: 0x0000 0000

Bit	31	30	29	28	27	26	25	24
Symbol	EP15TX	EP15RX	EP14TX	EP14RX	EP13TX	EP13RX	EP12TX	EP12RX
Bit	23	22	21	20	19	18	17	16
Symbol	EP11TX	EP11RX	EP10TX	EP10RX	EP9TX	EP9RX	EP8TX	EP8RX
Bit	15	14	13	12	11	10	9	8
Symbol	EP7TX	EP7RX	EP6TX	EP6RX	EP5TX	EP5RX	EP4TX	EP4RX
Bit	7	6	5	4	3	2	1	0
Symbol	EP3TX	EP3RX	EP2TX	EP2RX	EP1TX	EP1RX	EP0TX	EP0RX

Table 205. USB Endpoint Interrupt Enable register (USBEPIntEn - address 0x5000 C234) bit description

Bit	Symbol	Value	Description	Reset value
31:0	See USBEPIntEn bit allocation table above	0	The corresponding bit in USBDMARSt is set when an interrupt occurs for this endpoint.	0
		1	The corresponding bit in USBEPIntSt is set when an interrupt occurs for this endpoint. Implies Slave mode for this endpoint.	

11.10.3.3 USB Endpoint Interrupt Clear register (USBEPIntClr - 0x5000 C238)

Writing a one to this a bit in this register causes the SIE Select Endpoint/Clear Interrupt command to be executed ([Table 249](#)) for the corresponding physical endpoint. Writing zero has no effect. Before executing the Select Endpoint/Clear Interrupt command, the CDFULL bit in USBDevIntSt is cleared by hardware. On completion of the command, the CDFULL bit is set, USBCmdData contains the status of the endpoint, and the corresponding bit in USBEPIntSt is cleared.

Notes:

- When clearing interrupts using USBEPIntClr, software should wait for CDFULL to be set to ensure the corresponding interrupt has been cleared before proceeding.
- While setting multiple bits in USBEPIntClr simultaneously is possible, it is not recommended; only the status of the endpoint corresponding to the least significant interrupt bit cleared will be available at the end of the operation.
- Alternatively, the SIE Select Endpoint/Clear Interrupt command can be directly invoked using the SIE command registers, but using USBEPIntClr is recommended because of its ease of use.

Each physical endpoint has its own reserved bit in this register. The bit field definition is the same as that of USBEPIntSt shown in [Table 202](#). USBEPIntClr is a write-only register.

Table 206. USB Endpoint Interrupt Clear register (USBEpIntClr - address 0x5000 C238) bit allocation

Reset value: 0x0000 0000

Bit	31	30	29	28	27	26	25	24
Symbol	EP15TX	EP15RX	EP14TX	EP14RX	EP13TX	EP13RX	EP12TX	EP12RX
Bit	23	22	21	20	19	18	17	16
Symbol	EP11TX	EP11RX	EP10TX	EP10RX	EP9TX	EP9RX	EP8TX	EP8RX
Bit	15	14	13	12	11	10	9	8
Symbol	EP7TX	EP7RX	EP6TX	EP6RX	EP5TX	EP5RX	EP4TX	EP4RX
Bit	7	6	5	4	3	2	1	0
Symbol	EP3TX	EP3RX	EP2TX	EP2RX	EP1TX	EP1RX	EP0TX	EP0RX

Table 207. USB Endpoint Interrupt Clear register (USBEpIntClr - address 0x5000 C238) bit description

Bit	Symbol	Value	Description	Reset value
31:0	See USBEpIntClr bit allocation table above	0	No effect.	0
		1	Clears the corresponding bit in USBEpIntSt, by executing the SIE Select Endpoint/Clear Interrupt command for this endpoint.	

11.10.3.4 USB Endpoint Interrupt Set register (USBEpIntSet - 0x5000 C23C)

Writing a one to a bit in this register sets the corresponding bit in USBEpIntSt. Writing zero has no effect. Each endpoint has its own bit in this register. USBEpIntSet is a write-only register.

Table 208. USB Endpoint Interrupt Set register (USBEpIntSet - address 0x5000 C23C) bit allocation

Reset value: 0x0000 0000

Bit	31	30	29	28	27	26	25	24
Symbol	EP15TX	EP15RX	EP14TX	EP14RX	EP13TX	EP13RX	EP12TX	EP12RX
Bit	23	22	21	20	19	18	17	16
Symbol	EP11TX	EP11RX	EP10TX	EP10RX	EP9TX	EP9RX	EP8TX	EP8RX
Bit	15	14	13	12	11	10	9	8
Symbol	EP7TX	EP7RX	EP6TX	EP6RX	EP5TX	EP5RX	EP4TX	EP4RX
Bit	7	6	5	4	3	2	1	0
Symbol	EP3TX	EP3RX	EP2TX	EP2RX	EP1TX	EP1RX	EP0TX	EP0RX

Table 209. USB Endpoint Interrupt Set register (USBEpIntSet - address 0x5000 C23C) bit description

Bit	Symbol	Value	Description	Reset value
31:0	See USBEpIntSet bit allocation table above	0	No effect.	0
		1	Sets the corresponding bit in USBEpIntSt.	

11.10.3.5 USB Endpoint Interrupt Priority register (USBEpIntPri - 0x5000 C240)

This register determines whether an endpoint interrupt is routed to the EP_FAST or EP_SLOW bits of USBDevIntSt. If a bit in this register is set to one, the interrupt is routed to EP_FAST, if zero it is routed to EP_SLOW. Routing of multiple endpoints to EP_FAST or EP_SLOW is possible.

Note that the USBDevIntPri register determines whether the EP_FAST interrupt is routed to the USB_INT_REQ_HP or USB_INT_REQ_LP interrupt line.

USBEpIntPri is a write-only register.

Table 210. USB Endpoint Interrupt Priority register (USBEpIntPri - address 0x5000 C240) bit allocation

Reset value: 0x0000 0000

Bit	31	30	29	28	27	26	25	24
Symbol	EP15TX	EP15RX	EP14TX	EP14RX	EP13TX	EP13RX	EP12TX	EP12RX
Bit	23	22	21	20	19	18	17	16
Symbol	EP11TX	EP11RX	EP10TX	EP10RX	EP9TX	EP9RX	EP8TX	EP8RX
Bit	15	14	13	12	11	10	9	8
Symbol	EP7TX	EP7RX	EP6TX	EP6RX	EP5TX	EP5RX	EP4TX	EP4RX
Bit	7	6	5	4	3	2	1	0
Symbol	EP3TX	EP3RX	EP2TX	EP2RX	EP1TX	EP1RX	EP0TX	EP0RX

Table 211. USB Endpoint Interrupt Priority register (USBEpIntPri - address 0x5000 C240) bit description

Bit	Symbol	Value	Description	Reset value
31:0	See USBEpIntPri bit allocation table above	0	The corresponding interrupt is routed to the EP_SLOW bit of USBDevIntSt	0
		1	The corresponding interrupt is routed to the EP_FAST bit of USBDevIntSt	

11.10.4 Endpoint realization registers

The registers in this group allow realization and configuration of endpoints at run time.

11.10.4.1 EP RAM requirements

The USB device controller uses a RAM based FIFO for each endpoint buffer. The RAM dedicated for this purpose is called the Endpoint RAM (EP_RAM). Each endpoint has space reserved in the EP_RAM. The EP_RAM space required for an endpoint depends on its MaxPacketSize and whether it is double buffered. 32 words of EP_RAM are used by the device for storing the endpoint buffer pointers. The EP_RAM is word aligned but the MaxPacketSize is defined in bytes hence the RAM depth has to be adjusted to the next word boundary. Also, each buffer has one word header showing the size of the packet length received.

The EP_RAM space (in words) required for the physical endpoint can be expressed as

$$EPRAMspace = \left(\frac{MaxPacketSize + 3}{4} + 1 \right) \times dbstatus \quad (1)$$

where dbstatus = 1 for a single-buffered endpoint and 2 for a double-buffered endpoint.

Since all the realized endpoints occupy EP_RAM space, the total EP_RAM requirement is

$$TotalEPRAMspace = 32 + \sum_{n=0}^N EPRAMspace(n) \quad (2)$$

where N is the number of realized endpoints. Total EP_RAM space should not exceed 4096 bytes (4 kB, 1 kwords).

11.10.4.2 USB Realize Endpoint register (USBReEp - 0x5000 C244)

Writing one to a bit in this register causes the corresponding endpoint to be realized. Writing zeros causes it to be unrealized. This register returns to its reset state when a bus reset occurs. USBReEp is a read/write register.

Table 212. USB Realize Endpoint register (USBReEp - address 0x5000 C244) bit allocation

Reset value: 0x0000 0003

Bit	31	30	29	28	27	26	25	24
Symbol	EP31	EP30	EP29	EP28	EP27	EP26	EP25	EP24
Bit	23	22	21	20	19	18	17	16
Symbol	EP23	EP22	EP21	EP20	EP19	EP18	EP17	EP16
Bit	15	14	13	12	11	10	9	8
Symbol	EP15	EP14	EP13	EP12	EP11	EP10	EP9	EP8
Bit	7	6	5	4	3	2	1	0
Symbol	EP7	EP6	EP5	EP4	EP3	EP2	EP1	EP0

Table 213. USB Realize Endpoint register (USBReEp - address 0x5000 C244) bit description

Bit	Symbol	Value	Description	Reset value
0	EP0	0	Control endpoint EP0 is not realized.	1
		1	Control endpoint EP0 is realized.	
1	EP1	0	Control endpoint EP1 is not realized.	1
		1	Control endpoint EP1 is realized.	
31:2	EPxx	0	Endpoint EPxx is not realized.	0
		1	Endpoint EPxx is realized.	

On reset, only the control endpoints are realized. Other endpoints, if required, are realized by programming the corresponding bits in USBReEp. To calculate the required EP_RAM space for the realized endpoints, see [Section 11.10.4.1](#).

Realization of endpoints is a multi-cycle operation. Pseudo code for endpoint realization is shown below.

```
Clear EP_RLZED bit in USBDevIntSt;
```

```
for every endpoint to be realized,
{
```

```
    /* OR with the existing value of the Realize Endpoint register */
    USBReEp |= (UInt32) ((0x1 << endpt));
    /* Load Endpoint index Reg with physical endpoint no.*/
    USBEpIn = (UInt32) endpointnumber;
```

```
    /* load the max packet size Register */
    USBEpMaxPSize = MPS;
```

```
    /* check whether the EP_RLZED bit in the Device Interrupt Status register is set
    */
    while (!(USBDevIntSt & EP_RLZED))
    {
        /* wait until endpoint realization is complete */
    }
```

```

    }
    /* Clear the EP_RLZED bit */
    Clear EP_RLZED bit in USBDevIntSt;
}

```

The device will not respond to any transactions to unrealized endpoints. The SIE Configure Device command will only cause realized and enabled endpoints to respond to transactions. For details see [Table 244](#).

11.10.4.3 USB Endpoint Index register (USBEPIn - 0x5000 C248)

Each endpoint has a register carrying the MaxPacketSize value for that endpoint. This is in fact a register array. Hence before writing, this register is addressed through the USBEPIn register.

The USBEPIn register will hold the physical endpoint number. Writing to USBMaxPSize will set the array element pointed to by USBEPIn. USBEPIn is a write-only register.

Table 214. USB Endpoint Index register (USBEPIn - address 0x5000 C248) bit description

Bit	Symbol	Description	Reset value
4:0	PHY_EP	Physical endpoint number (0-31)	0
31:5	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

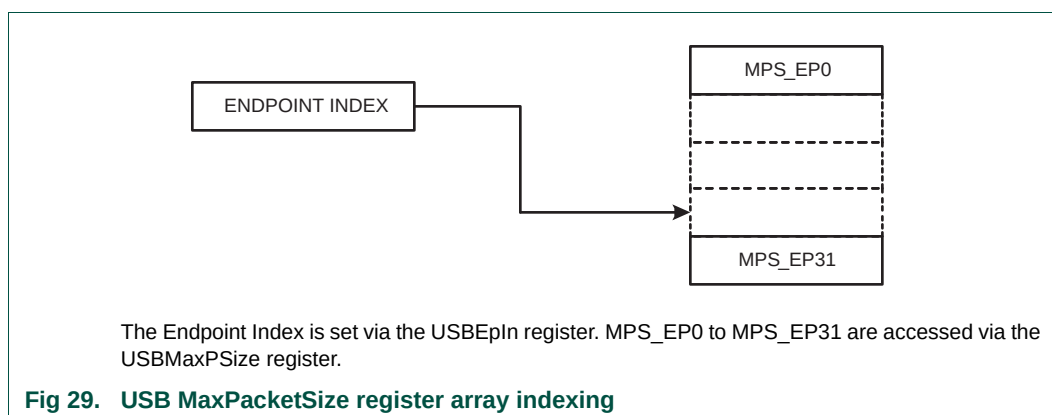
11.10.4.4 USB MaxPacketSize register (USBMaxPSize - 0x5000 C24C)

On reset, the control endpoint is assigned the maximum packet size of 8 bytes. Other endpoints are assigned 0. Modifying USBMaxPSize will cause the endpoint buffer addresses within the EP_RAM to be recalculated. This is a multi-cycle process. At the end, the EP_RLZED bit will be set in USBDevIntSt ([Table 193](#)). USBMaxPSize array indexing is shown in [Figure 29](#). USBMaxPSize is a read/write register.

Table 215. USB MaxPacketSize register (USBMaxPSize - address 0x5000 C24C) bit description

Bit	Symbol	Description	Reset value
9:0	MPS	The maximum packet size value.	0x008 ^[1]
31:10	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

[1] Reset value for EP0 and EP1. All other endpoints have a reset value of 0x0.



11.10.5 USB transfer registers

The registers in this group are used for transferring data between endpoint buffers and RAM in Slave mode operation. See [Section 11.14 “Slave mode operation”](#).

11.10.5.1 USB Receive Data register (USBRxData - 0x5000 C218)

For an OUT transaction, the CPU reads the endpoint buffer data from this register. Before reading this register, the RD_EN bit and LOG_ENDPOINT field of the USBCtrl register should be set appropriately. On reading this register, data from the selected endpoint buffer is fetched. The data is in little endian format: the first byte received from the USB bus will be available in the least significant byte of USBRxData. USBRxData is a read-only register.

Table 216. USB Receive Data register (USBRxData - address 0x5000 C218) bit description

Bit	Symbol	Description	Reset value
31:0	RX_DATA	Data received.	0x0000 0000

11.10.5.2 USB Receive Packet Length register (USBRxPLen - 0x5000 C220)

This register contains the number of bytes remaining in the endpoint buffer for the current packet being read via the USBRxData register, and a bit indicating whether the packet is valid or not. Before reading this register, the RD_EN bit and LOG_ENDPOINT field of the USBCtrl register should be set appropriately. This register is updated on each read of the USBRxData register. USBRxPLen is a read-only register.

Table 217. USB Receive Packet Length register (USBRxPLen - address 0x5000 C220) bit description

Bit	Symbol	Value	Description	Reset value
9:0	PKT_LNGTH	-	The remaining number of bytes to be read from the currently selected endpoint's buffer. When this field decrements to 0, the RxENDPKT bit will be set in USBDevIntSt.	0
10	DV		Data valid. This bit is useful for isochronous endpoints. Non-isochronous endpoints do not raise an interrupt when an erroneous data packet is received. But invalid data packet can be produced with a bus reset. For isochronous endpoints, data transfer will happen even if an erroneous packet is received. In this case DV bit will not be set for the packet.	0
		0	Data is invalid.	
		1	Data is valid.	
11	PKT_RDY	-	The PKT_LNGTH field is valid and the packet is ready for reading.	0
31:12	-	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

11.10.5.3 USB Transmit Data register (USBTxData - 0x5000 C21C)

For an IN transaction, the CPU writes the endpoint data into this register. Before writing to this register, the WR_EN bit and LOG_ENDPOINT field of the USBCtrl register should be set appropriately, and the packet length should be written to the USBTxPlen register. On writing this register, the data is written to the selected endpoint buffer. The data is in little endian format: the first byte sent on the USB bus will be the least significant byte of USBTxData. USBTxData is a write-only register.

Table 218. USB Transmit Data register (USBTxData - address 0x5000 C21C) bit description

Bit	Symbol	Description	Reset value
31:0	TX_DATA	Transmit Data.	0x0000 0000

11.10.5.4 USB Transmit Packet Length register (USBTxPLen - 0x5000 C224)

This register contains the number of bytes transferred from the CPU to the selected endpoint buffer. Before writing data to USBTxData, software should first write the packet length ($\leq$ MaxPacketSize) to this register. After each write to USBTxData, hardware decrements USBTxPLen by 4. The WR_EN bit and LOG_ENDPOINT field of the USBCtrl register should be set to select the desired endpoint buffer before starting this process.

For data buffers larger than the endpoint's MaxPacketSize, software should submit data in packets of MaxPacketSize, and send the remaining extra bytes in the last packet. For example, if the MaxPacketSize is 64 bytes and the data buffer to be transferred is of length 130 bytes, then the software sends two 64-byte packets and the remaining 2 bytes in the last packet. So, a total of 3 packets are sent on USB. USBTxPLen is a write-only register.

Table 219. USB Transmit Packet Length register (USBTxPLen - address 0x5000 C224) bit description

Bit	Symbol	Value	Description	Reset value
9:0	PKT_LNGTH	-	The remaining number of bytes to be written to the selected endpoint buffer. This field is decremented by 4 by hardware after each write to USBTxData. When this field decrements to 0, the TxENDPKT bit will be set in USBDevIntSt.	0x000
31:10	-	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

11.10.5.5 USB Control register (USBCtrl - 0x5000 C228)

This register controls the data transfer operation of the USB device. It selects the endpoint buffer that is accessed by the USBRxData and USBTxData registers, and enables reading and writing them. USBCtrl is a read/write register.

Table 220. USB Control register (USBCtrl - address 0x5000 C228) bit description

Bit	Symbol	Value	Description	Reset value
0	RD_EN		Read mode control. Enables reading data from the OUT endpoint buffer for the endpoint specified in the LOG_ENDPOINT field using the USBRxData register. This bit is cleared by hardware when the last word of the current packet is read from USBRxData.	0
		0	Read mode is disabled.	
		1	Read mode is enabled.	
1	WR_EN		Write mode control. Enables writing data to the IN endpoint buffer for the endpoint specified in the LOG_ENDPOINT field using the USBTxData register. This bit is cleared by hardware when the number of bytes in USBTxLen have been sent.	0
		0	Write mode is disabled.	
		1	Write mode is enabled.	
5:2	LOG_ENDPOINT	-	Logical Endpoint number.	0x0
31:6	-	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

11.10.6 SIE command code registers

The SIE command code registers are used for communicating with the Serial Interface Engine. See [Section 11.12 “Serial interface engine command description”](#) for more information.

11.10.6.1 USB Command Code register (USBCmdCode - 0x5000 C210)

This register is used for sending the command and write data to the SIE. The commands written here are propagated to the SIE and executed there. After executing the command, the register is empty, and the CCEMPTY bit of USBDevIntSt register is set. See [Section 11.12](#) for details. USBCmdCode is a write-only register.

Table 221. USB Command Code register (USBCmdCode - address 0x5000 C210) bit description

Bit	Symbol	Value	Description	Reset value
7:0	-	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA
15:8	CMD_PHASE		The command phase:	0x00
		0x01	Write	
		0x02	Read	
		0x05	Command	
23:16	CMD_CODE/ CMD_WDATA		This is a multi-purpose field. When CMD_PHASE is Command or Read, this field contains the code for the command (CMD_CODE). When CMD_PHASE is Write, this field contains the command write data (CMD_WDATA).	0x00
31:24	-	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

11.10.6.2 USB Command Data register (USBCmdData - 0x5000 C214)

This register contains the data retrieved after executing a SIE command. When the data is ready to be read, the CD_FULL bit of the USBDevIntSt register is set. See [Table 193](#) for details. USBCmdData is a read-only register.

Table 222. USB Command Data register (USBCmdData - address 0x5000 C214) bit description

Bit	Symbol	Description	Reset value
7:0	CMD_RDATA	Command Read Data.	0x00
31:8	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

11.10.7 DMA registers

The registers in this group are used for the DMA mode of operation (see [Section 11.15 “DMA operation”](#))

11.10.7.1 USB DMA Request Status register (USBDMARSt - 0x5000 C250)

A bit in this register associated with a non-isochronous endpoint is set by hardware when an endpoint interrupt occurs (see the description of USBEpIntSt) and the corresponding bit in USBEpIntEn is 0. A bit associated with an isochronous endpoint is set when the corresponding bit in USBEpIntEn is 0 and a FRAME interrupt occurs. A set bit serves as a

flag for the DMA engine to start the data transfer if the DMA is enabled for the corresponding endpoint in the USBEPDMARSt register. The DMA cannot be enabled for control endpoints (EP0 and EP1). USBDMARSt is a read-only register.

Table 223. USB DMA Request Status register (USBDMARSt - address 0x5000 C250) bit allocation

Reset value: 0x0000 0000

Bit	31	30	29	28	27	26	25	24
Symbol	EP31	EP30	EP29	EP28	EP27	EP26	EP25	EP24
Bit	23	22	21	20	19	18	17	16
Symbol	EP23	EP22	EP21	EP20	EP19	EP18	EP17	EP16
Bit	15	14	13	12	11	10	9	8
Symbol	EP15	EP14	EP13	EP12	EP11	EP10	EP9	EP8
Bit	7	6	5	4	3	2	1	0
Symbol	EP7	EP6	EP5	EP4	EP3	EP2	EP1	EP0

Table 224. USB DMA Request Status register (USBDMARSt - address 0x5000 C250) bit description

Bit	Symbol	Value	Description	Reset value
0	EP0	0	Control endpoint OUT (DMA cannot be enabled for this endpoint and EP0 bit must be 0).	0
1	EP1	0	Control endpoint IN (DMA cannot be enabled for this endpoint and EP1 bit must be 0).	0
31:2	EPxx		Endpoint xx ($2 \leq xx \leq 31$) DMA request.	0
		0	DMA not requested by endpoint xx.	
		1	DMA requested by endpoint xx.	

[1] DMA can not be enabled for this endpoint and the corresponding bit in the USBDMARSt must be 0.

11.10.7.2 USB DMA Request Clear register (USBDMARClr - 0x5000 C254)

Writing one to a bit in this register will clear the corresponding bit in the USBDMARSt register. Writing zero has no effect.

This register is intended for initialization prior to enabling the DMA for an endpoint. When the DMA is enabled for an endpoint, hardware clears the corresponding bit in USBDMARSt on completion of a packet transfer. Therefore, software should not clear the bit using this register while the endpoint is enabled for DMA operation.

USBDMARClr is a write-only register.

The USBDMARClr bit allocation is identical to the USBDMARSt register ([Table 223](#)).

Table 225. USB DMA Request Clear register (USBDMARClr - address 0x5000 C254) bit description

Bit	Symbol	Value	Description	Reset value
0	EP0	0	Control endpoint OUT (DMA cannot be enabled for this endpoint and the EP0 bit must be 0).	0
1	EP1	0	Control endpoint IN (DMA cannot be enabled for this endpoint and the EP1 bit must be 0).	0
31:2	EPxx		Clear the endpoint xx ($2 \leq xx \leq 31$) DMA request.	0
		0	No effect.	
		1	Clear the corresponding bit in USBDMARSt.	

11.10.7.3 USB DMA Request Set register (USBDMARSet - 0x5000 C258)

Writing one to a bit in this register sets the corresponding bit in the USBDMARSt register. Writing zero has no effect.

This register allows software to raise a DMA request. This can be useful when switching from Slave to DMA mode of operation for an endpoint: if a packet to be processed in DMA mode arrives before the corresponding bit of USBEpIntEn is cleared, the DMA request is not raised by hardware. Software can then use this register to manually start the DMA transfer.

Software can also use this register to initiate a DMA transfer to proactively fill an IN endpoint buffer before an IN token packet is received from the host.

USBDMARSet is a write-only register.

The USBDMARSet bit allocation is identical to the USBDMARSt register ([Table 223](#)).

Table 226. USB DMA Request Set register (USBDMARSet - address 0x5000 C258) bit description

Bit	Symbol	Value	Description	Reset value
0	EP0	0	Control endpoint OUT (DMA cannot be enabled for this endpoint and the EP0 bit must be 0).	0
1	EP1	0	Control endpoint IN (DMA cannot be enabled for this endpoint and the EP1 bit must be 0).	0
31:2	EPxx		Set the endpoint xx ($2 \leq xx \leq 31$) DMA request.	0
		0	No effect.	
		1	Set the corresponding bit in USBDMARSt.	

11.10.7.4 USB UDCA Head register (USBUDCAH - 0x5000 C280)

The UDCA (USB Device Communication Area) Head register maintains the address where the UDCA is located in the RAM. Refer to [Section 11.15.2 “USB device communication area”](#) and [Section 11.15.4 “The DMA descriptor”](#) for more details on the UDCA and DMA descriptors. USBUDCAH is a read/write register.

Table 227. USB UDCA Head register (USBUDCAH - address 0x5000 C280) bit description

Bit	Symbol	Description	Reset value
6:0	-	Reserved. Software should not write ones to reserved bits. The UDCA is aligned to 128-byte boundaries.	0x00
31:7	UDCA_ADDR	Start address of the UDCA.	0

11.10.7.5 USB EP DMA Status register (USBEPDMASt - 0x5000 C284)

Bits in this register indicate whether DMA operation is enabled for the corresponding endpoint. A DMA transfer for an endpoint can start only if the corresponding bit is set in this register. USBEPDMASt is a read-only register.

Table 228. USB EP DMA Status register (USBEPDMASt - address 0x5000 C284) bit description

Bit	Symbol	Value	Description	Reset value
0	EP0_DMA_ENABLE	0	Control endpoint OUT (DMA cannot be enabled for this endpoint and the EP0_DMA_ENABLE bit must be 0).	0
1	EP1_DMA_ENABLE	0	Control endpoint IN (DMA cannot be enabled for this endpoint and the EP1_DMA_ENABLE bit must be 0).	0
31:2	EPxx_DMA_ENABLE		endpoint xx ($2 \leq xx \leq 31$) DMA enabled bit.	0
		0	The DMA for endpoint EPxx is disabled.	
		1	The DMA for endpoint EPxx is enabled.	

11.10.7.6 USB EP DMA Enable register (USBEPDMAEn - 0x5000 C288)

Writing one to a bit to this register will enable the DMA operation for the corresponding endpoint. Writing zero has no effect. The DMA cannot be enabled for control endpoints EP0 and EP1. USBEPDMAEn is a write-only register.

Table 229. USB EP DMA Enable register (USBEPDMAEn - address 0x5000 C288) bit description

Bit	Symbol	Value	Description	Reset value
0	EP0_DMA_ENABLE	0	Control endpoint OUT (DMA cannot be enabled for this endpoint and the EP0_DMA_ENABLE bit value must be 0).	0
1	EP1_DMA_ENABLE	0	Control endpoint IN (DMA cannot be enabled for this endpoint and the EP1_DMA_ENABLE bit must be 0).	0
31:2	EPxx_DMA_ENABLE		Endpoint xx ($2 \leq xx \leq 31$) DMA enable control bit.	0
		0	No effect.	
		1	Enable the DMA operation for endpoint EPxx.	

11.10.7.7 USB EP DMA Disable register (USBEPDMADis - 0x5000 C28C)

Writing a one to a bit in this register clears the corresponding bit in USBEPDMASt. Writing zero has no effect on the corresponding bit of USBEPDMASt. Any write to this register clears the internal DMA_PROCEED flag. Refer to [Section 11.15.5.4 "Optimizing descriptor fetch"](#) for more information on the DMA_PROCEED flag. If a DMA transfer is in progress for an endpoint when its corresponding bit is cleared, the transfer is completed before the DMA is disabled. When an error condition is detected during a DMA transfer, the corresponding bit is cleared by hardware. USBEPDMADis is a write-only register.

Table 230. USB EP DMA Disable register (USBEPDMADis - address 0x5000 C28C) bit description

Bit	Symbol	Value	Description	Reset value
0	EP0_DMA_DISABLE	0	Control endpoint OUT (DMA cannot be enabled for this endpoint and the EP0_DMA_DISABLE bit value must be 0).	0
1	EP1_DMA_DISABLE	0	Control endpoint IN (DMA cannot be enabled for this endpoint and the EP1_DMA_DISABLE bit value must be 0).	0
31:2	EPxx_DMA_DISABLE		Endpoint xx ($2 \leq xx \leq 31$) DMA disable control bit.	0
		0	No effect.	
		1	Disable the DMA operation for endpoint EPxx.	

11.10.7.8 USB DMA Interrupt Status register (USBDMAIntSt - 0x5000 C290)

Each bit of this register reflects whether any of the 32 bits in the corresponding interrupt status register are set. USBDMAIntSt is a read-only register.

Table 231. USB DMA Interrupt Status register (USBDMIntSt - address 0x5000 C290) bit description

Bit	Symbol	Value	Description	Reset value
0	EOT		End of Transfer Interrupt bit.	0
		0	All bits in the USBEoTIntSt register are 0.	
		1	At least one bit in the USBEoTIntSt is set.	
1	NDDR		New DD Request Interrupt bit.	0
		0	All bits in the USBNDDRIntSt register are 0.	
		1	At least one bit in the USBNDDRIntSt is set.	
2	ERR		System Error Interrupt bit.	0
		0	All bits in the USBSysErrIntSt register are 0.	
		1	At least one bit in the USBSysErrIntSt is set.	
31:3	-	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

11.10.7.9 USB DMA Interrupt Enable register (USBDMIntEn - 0x5000 C294)

Writing a one to a bit in this register enables the corresponding bit in USBDMIntSt to generate an interrupt on the USB_INT_REQ_DMA interrupt line when set. USBDMIntEn is a read/write register.

Table 232. USB DMA Interrupt Enable register (USBDMIntEn - address 0x5000 C294) bit description

Bit	Symbol	Value	Description	Reset value
0	EOT		End of Transfer Interrupt enable bit.	0
		0	The End of Transfer Interrupt is disabled.	
		1	The End of Transfer Interrupt is enabled.	
1	NDDR		New DD Request Interrupt enable bit.	0
		0	The New DD Request Interrupt is disabled.	
		1	The New DD Request Interrupt is enabled.	
2	ERR		System Error Interrupt enable bit.	0
		0	The System Error Interrupt is disabled.	
		1	The System Error Interrupt is enabled.	
31:3	-	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

11.10.7.10 USB End of Transfer Interrupt Status register (USBEOtIntSt - 0x5000 C2A0)

When the DMA transfer completes for the current DMA descriptor, either normally (descriptor is retired) or because of an error, the bit corresponding to the endpoint is set in this register. The cause of the interrupt is recorded in the DD_status field of the descriptor. USBEOtIntSt is a read-only register.

Table 233. USB End of Transfer Interrupt Status register (USBEOtIntSt - address 0x5000 C2A0s) bit description

Bit	Symbol	Value	Description	Reset value
31:0	EPxx		Endpoint xx ($2 \leq xx \leq 31$) End of Transfer Interrupt request.	0
		0	There is no End of Transfer interrupt request for endpoint xx.	
		1	There is an End of Transfer Interrupt request for endpoint xx.	

11.10.7.11 USB End of Transfer Interrupt Clear register (USBEoTIntClr - 0x5000 C2A4)

Writing one to a bit in this register clears the corresponding bit in the USBEoTIntSt register. Writing zero has no effect. USBEoTIntClr is a write-only register.

Table 234. USB End of Transfer Interrupt Clear register (USBEoTIntClr - address 0x5000 C2A4) bit description

Bit	Symbol	Value	Description	Reset value
31:0	EPxx		Clear endpoint xx ($2 \leq xx \leq 31$) End of Transfer Interrupt request.	0
		0	No effect.	
		1	Clear the EPxx End of Transfer Interrupt request in the USBEoTIntSt register.	

11.10.7.12 USB End of Transfer Interrupt Set register (USBEoTIntSet - 0x5000 C2A8)

Writing one to a bit in this register sets the corresponding bit in the USBEoTIntSt register. Writing zero has no effect. USBEoTIntSet is a write-only register.

Table 235. USB End of Transfer Interrupt Set register (USBEoTIntSet - address 0x5000 C2A8) bit description

Bit	Symbol	Value	Description	Reset value
31:0	EPxx		Set endpoint xx ($2 \leq xx \leq 31$) End of Transfer Interrupt request.	0
		0	No effect.	
		1	Set the EPxx End of Transfer Interrupt request in the USBEoTIntSt register.	

11.10.7.13 USB New DD Request Interrupt Status register (USBNDDRIntSt - 0x5000 C2AC)

A bit in this register is set when a transfer is requested from the USB device and no valid DD is detected for the corresponding endpoint. USBNDDRIntSt is a read-only register.

Table 236. USB New DD Request Interrupt Status register (USBNDDRIntSt - address 0x5000 C2AC) bit description

Bit	Symbol	Value	Description	Reset value
31:0	EPxx		Endpoint xx ($2 \leq xx \leq 31$) new DD interrupt request.	0
		0	There is no new DD interrupt request for endpoint xx.	
		1	There is a new DD interrupt request for endpoint xx.	

11.10.7.14 USB New DD Request Interrupt Clear register (USBNDDRIntClr - 0x5000 C2B0)

Writing one to a bit in this register clears the corresponding bit in the USBNDDRIntSt register. Writing zero has no effect. USBNDDRIntClr is a write-only register.

Table 237. USB New DD Request Interrupt Clear register (USBNDDRIntClr - address 0x5000 C2B0) bit description

Bit	Symbol	Value	Description	Reset value
31:0	EPxx		Clear endpoint xx ($2 \leq xx \leq 31$) new DD interrupt request.	0
		0	No effect.	
		1	Clear the EPxx new DD interrupt request in the USBNDDRIntSt register.	

11.10.7.15 USB New DD Request Interrupt Set register (USBNDDRIntSet - 0x5000 C2B4)

Writing one to a bit in this register sets the corresponding bit in the USBNDDRIntSt register. Writing zero has no effect. USBNDDRIntSet is a write-only register.

Table 238. USB New DD Request Interrupt Set register (USBNDDRIntSet - address 0x5000 C2B4) bit description

Bit	Symbol	Value	Description	Reset value
31:0	EPxx		Set endpoint xx ($2 \leq xx \leq 31$) new DD interrupt request.	0
		0	No effect.	
		1	Set the EPxx new DD interrupt request in the USBNDDRIntSt register.	

11.10.7.16 USB System Error Interrupt Status register (USBSysErrIntSt - 0x5000 C2B8)

If a system error (AHB bus error) occurs when transferring the data or when fetching or updating the DD the corresponding bit is set in this register. USBSysErrIntSt is a read-only register.

Table 239. USB System Error Interrupt Status register (USBSysErrIntSt - address 0x5000 C2B8) bit description

Bit	Symbol	Value	Description	Reset value
31:0	EPxx		Endpoint xx ($2 \leq xx \leq 31$) System Error Interrupt request.	0
		0	There is no System Error Interrupt request for endpoint xx.	
		1	There is a System Error Interrupt request for endpoint xx.	

11.10.7.17 USB System Error Interrupt Clear register (USBSysErrIntClr - 0x5000 C2BC)

Writing one to a bit in this register clears the corresponding bit in the USBSysErrIntSt register. Writing zero has no effect. USBSysErrIntClr is a write-only register.

Table 240. USB System Error Interrupt Clear register (USBSysErrIntClr - address 0x5000 C2BC) bit description

Bit	Symbol	Value	Description	Reset value
31:0	EPxx		Clear endpoint xx ($2 \leq xx \leq 31$) System Error Interrupt request.	0
		0	No effect.	
		1	Clear the EPxx System Error Interrupt request in the USBSysErrIntSt register.	

11.10.7.18 USB System Error Interrupt Set register (USBSysErrIntSet - 0x5000 C2C0)

Writing one to a bit in this register sets the corresponding bit in the USBSysErrIntSt register. Writing zero has no effect. USBSysErrIntSet is a write-only register.

Table 241. USB System Error Interrupt Set register (USBSysErrIntSet - address 0x5000 C2C0) bit description

Bit	Symbol	Value	Description	Reset value
31:0	EPxx		Set endpoint xx ($2 \leq xx \leq 31$) System Error Interrupt request.	0
		0	No effect.	
		1	Set the EPxx System Error Interrupt request in the USBSysErrIntSt register.	

11.11 Interrupt handling

This section describes how an interrupt event on any of the endpoints is routed to the Nested Vectored Interrupt Controller (NVIC). For a diagram showing interrupt event handling, see [Figure 30](#).

All non-isochronous OUT endpoints (control, bulk, and interrupt endpoints) generate an interrupt when they receive a packet without an error. All non-isochronous IN endpoints generate an interrupt when a packet has been successfully transmitted or when a NAK signal is sent and interrupts on NAK are enabled by the SIE Set Mode command, see [Section 11.12.3](#). For isochronous endpoints, a frame interrupt is generated every 1 ms.

The interrupt handling is different for Slave and DMA mode.

Slave mode

If an interrupt event occurs on an endpoint and the endpoint interrupt is enabled in the USBEpIntEn register, the corresponding status bit in the USBEpIntSt is set. For non-isochronous endpoints, all endpoint interrupt events are divided into two types by the corresponding USBEpIntPri[n] registers: fast endpoint interrupt events and slow endpoint interrupt events. All fast endpoint interrupt events are ORed and routed to bit EP_FAST in the USBDevIntSt register. All slow endpoint interrupt events are ORed and routed to the EP_SLOW bit in USBDevIntSt.

For isochronous endpoints, the FRAME bit in USBDevIntSt is set every 1 ms.

The USBDevIntSt register holds the status of all endpoint interrupt events as well as the status of various other interrupts (see [Section 11.10.2.2](#)). By default, all interrupts (if enabled in USBDevIntEn) are routed to the USB_INT_REQ_LP bit in the USBIntSt register to request low priority interrupt handling. However, the USBDevIntPri register can route either the FRAME or the EP_FAST bit to the USB_INT_REQ_HP bit in the USBIntSt register.

Only one of the EP_FAST and FRAME interrupt events can be routed to the USB_INT_REQ_HP bit. If routing both bits to USB_INT_REQ_HP is attempted, both interrupt events are routed to USB_INT_REQ_LP.

Slow endpoint interrupt events are always routed directly to the USB_INT_REQ_LP bit for low priority interrupt handling by software.

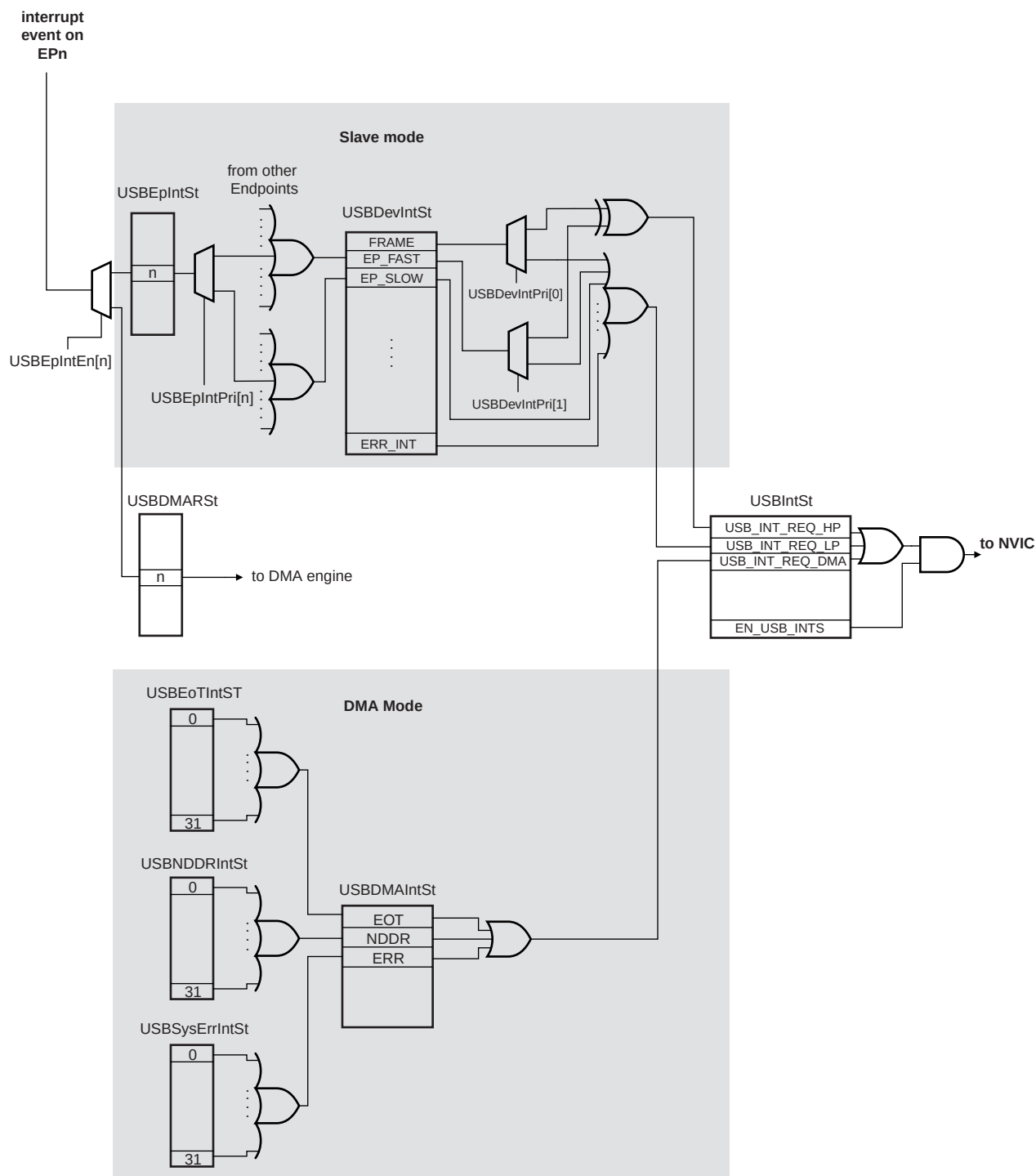
The final interrupt signal to the NVIC is gated by the EN_USB_INTS bit in the USBIntSt register. The USB interrupts are routed to the NVIC only if EN_USB_INTS is set.

DMA mode

If an interrupt event occurs on a non-control endpoint and the endpoint interrupt is not enabled in the USBEpIntEn register, the corresponding status bit in the USBDMARSt is set by hardware. This serves as a flag for the DMA engine to transfer data if DMA transfer is enabled for the corresponding endpoint in the USBEpDMASt register.

Three types of interrupts can occur for each endpoint for data transfers in DMA mode: End of transfer interrupt, new DD request interrupt, and system error interrupt. These interrupt events set a bit for each endpoint in the respective registers USBEoTIntSt, USBNDDRIntSt, and USBSysErrIntSt. The End of transfer interrupts from all endpoints are then ORed and routed to the EOT bit in USBDMAIntSt. Likewise, all New DD request interrupts and system error interrupt events are routed to the NDDR and ERR bits respectively in the USBDMAStInt register.

The EOT, NDDR, and ERR bits (if enabled in USBDMAIntEn) are ORed to set the USB_INT_REQ_DMA bit in the USBIntSt register. If the EN_USB_INTS bit is set in USBIntSt, the interrupt is routed to the NVIC.



For simplicity, USBDevIntEn and USBDMAIntEn are not shown.

Fig 30. Interrupt event handling

11.12 Serial interface engine command description

The functions and registers of the Serial Interface Engine (SIE) are accessed using commands, which consist of a command code followed by optional data bytes (read or write action). The USBCmdCode ([Table 221](#)) and USBCmdData ([Table 222](#)) registers are used for these accesses.

A complete access consists of two phases:

1. **Command phase:** the USBCmdCode register is written with the CMD_PHASE field set to the value 0x05 (Command), and the CMD_CODE field set to the desired command code. On completion of the command, the CCEMPTY bit of USBDevIntSt is set.
2. **Data phase (optional):** for writes, the USBCmdCode register is written with the CMD_PHASE field set to the value 0x01 (Write), and the CMD_WDATA field set with the desired write data. On completion of the write, the CCEMPTY bit of USBDevIntSt is set. For reads, USBCmdCode register is written with the CMD_PHASE field set to the value 0x02 (Read), and the CMD_CODE field set with command code the read corresponds to. On completion of the read, the CDFULL bit of USBDevIntSt will be set, indicating the data is available for reading in the USBCmdData register. In the case of multi-byte registers, the least significant byte is accessed first.

An overview of the available commands is given in [Table 242](#).

Here is an example of the Read Current Frame Number command (reading 2 bytes):

```

USBDevIntClr = 0x30;           // Clear both CCEMPTY & CDFULL
USBCmdCode = 0x00F50500;      // CMD_CODE=0xF5, CMD_PHASE=0x05(Command)
while (!(USBDevIntSt & 0x10)); // Wait for CCEMPTY.
USBDevIntClr = 0x10;          // Clear CCEMPTY interrupt bit.
USBCmdCode = 0x00F50200;      // CMD_CODE=0xF5, CMD_PHASE=0x02(Read)
while (!(USBDevIntSt & 0x20)); // Wait for CDFULL.
USBDevIntClr = 0x20;          // Clear CDFULL.
CurFrameNum = USBCmdData;     // Read Frame number LSB byte.
USBCmdCode = 0x00F50200;      // CMD_CODE=0xF5, CMD_PHASE=0x02(Read)
while (!(USBDevIntSt & 0x20)); // Wait for CDFULL.
Temp = USBCmdData;            // Read Frame number MSB byte
USBDevIntClr = 0x20;          // Clear CDFULL interrupt bit.
CurFrameNum = CurFrameNum | (Temp << 8);

```

Here is an example of the Set Address command (writing 1 byte):

```

USBDevIntClr = 0x10;          // Clear CCEMPTY.
USBCmdCode = 0x00D00500;      // CMD_CODE=0xD0, CMD_PHASE=0x05(Command)
while (!(USBDevIntSt & 0x10)); // Wait for CCEMPTY.
USBDevIntClr = 0x10;          // Clear CCEMPTY.
USBCmdCode = 0x008A0100;      // CMD_WDATA=0x8A(DEV_EN=1, DEV_ADDR=0xA),
                                // CMD_PHASE=0x01(Write)
while (!(USBDevIntSt & 0x10)); // Wait for CCEMPTY.
USBDevIntClr = 0x10;          // Clear CCEMPTY.

```


Table 242. SIE command code table

Command name	Recipient	Code (Hex)	Data phase
Device commands			
Set Address	Device	D0	Write 1 byte
Configure Device	Device	D8	Write 1 byte
Set Mode	Device	F3	Write 1 byte
Read Current Frame Number	Device	F5	Read 1 or 2 bytes
Read Test Register	Device	FD	Read 2 bytes
Set Device Status	Device	FE	Write 1 byte
Get Device Status	Device	FE	Read 1 byte
Get Error Code	Device	FF	Read 1 byte
Read Error Status	Device	FB	Read 1 byte
Endpoint Commands			
Select Endpoint	Endpoint 0	00	Read 1 byte (optional)
	Endpoint 1	01	Read 1 byte (optional)
	Endpoint xx	xx	Read 1 byte (optional)
Select Endpoint/Clear Interrupt	Endpoint 0	40	Read 1 byte
	Endpoint 1	41	Read 1 byte
	Endpoint xx	xx + 40	Read 1 byte
Set Endpoint Status	Endpoint 0	40	Write 1 byte
	Endpoint 1	41	Write 1 byte
	Endpoint xx	xx + 40	Write 1 byte
Clear Buffer	Selected Endpoint	F2	Read 1 byte (optional)
Validate Buffer	Selected Endpoint	FA	None

11.12.1 Set Address (Command: 0xD0, Data: write 1 byte)

The Set Address command is used to set the USB assigned address and enable the (embedded) function. The address set in the device will take effect after the status stage of the control transaction. After a bus reset, DEV_ADDR is set to 0x00, and DEV_EN is set to 1. The device will respond to packets for function address 0x00, endpoint 0 (default endpoint).

Table 243. Set Address command bit description

Bit	Symbol	Description	Reset value
6:0	DEV_ADDR	Device address set by the software. After a bus reset this field is set to 0x00.	0x00
7	DEV_EN	Device Enable. After a bus reset this bit is set to 1. 0: Device will not respond to any packets. 1: Device will respond to packets for function address DEV_ADDR.	0

11.12.2 Configure Device (Command: 0xD8, Data: write 1 byte)

A value of 1 written to the register indicates that the device is configured and all the enabled non-control endpoints will respond. Control endpoints are always enabled and respond even if the device is not configured, in the default state.

Table 244. Configure Device command bit description

Bit	Symbol	Description	Reset value
0	CONF_DEVICE	Device is configured. All enabled non-control endpoints will respond. This bit is cleared by hardware when a bus reset occurs. When set, the UP_LED signal is driven LOW if the device is not in the suspended state (SUS=0).	
7:1	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

11.12.3 Set Mode (Command: 0xF3, Data: write 1 byte)

Table 245. Set Mode command bit description

Bit	Symbol	Value	Description	Reset value
0	AP_CLK		Always PLL Clock.	0
		0	USB_NEED_CLK is functional; the 48 MHz clock can be stopped when the device enters suspend state.	
		1	USB_NEED_CLK is fixed to 1; the 48 MHz clock cannot be stopped when the device enters suspend state.	
1	INAK_CI		Interrupt on NAK for Control IN endpoint.	0
		0	Only successful transactions generate an interrupt.	
		1	Both successful and NAKed IN transactions generate interrupts.	
2	INAK_CO		Interrupt on NAK for Control OUT endpoint.	0
		0	Only successful transactions generate an interrupt.	
		1	Both successful and NAKed OUT transactions generate interrupts.	
3	INAK_I1		Interrupt on NAK for Interrupt IN endpoint.	0
		0	Only successful transactions generate an interrupt.	
		1	Both successful and NAKed IN transactions generate interrupts.	
4	INAK_IO ^[1]		Interrupt on NAK for Interrupt OUT endpoints.	0
		0	Only successful transactions generate an interrupt.	
		1	Both successful and NAKed OUT transactions generate interrupts.	
5	INAK_B1		Interrupt on NAK for Bulk IN endpoints.	0
		0	Only successful transactions generate an interrupt.	
		1	Both successful and NAKed IN transactions generate interrupts.	
6	INAK_BO ^[2]		Interrupt on NAK for Bulk OUT endpoints.	0
		0	Only successful transactions generate an interrupt.	
		1	Both successful and NAKed OUT transactions generate interrupts.	
7	-	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

[1] This bit should be reset to 0 if the DMA is enabled for any of the Interrupt OUT endpoints.

[2] This bit should be reset to 0 if the DMA is enabled for any of the Bulk OUT endpoints.

11.12.4 Read Current Frame Number (Command: 0xF5, Data: read 1 or 2 bytes)

Returns the frame number of the last successfully received SOF. The frame number is eleven bits wide. The frame number returns least significant byte first. In case the user is only interested in the lower 8 bits of the frame number, only the first byte needs to be read.

- In case no SOF was received by the device at the beginning of a frame, the frame number returned is that of the last successfully received SOF.
- In case the SOF frame number contained a CRC error, the frame number returned will be the corrupted frame number as received by the device.

11.12.5 Read Test Register (Command: 0xFD, Data: read 2 bytes)

The test register is 16 bits wide. It returns the value of 0xA50F if the USB clocks (usbclk and AHB slave clock) are running.

11.12.6 Set Device Status (Command: 0xFE, Data: write 1 byte)

The Set Device Status command sets bits in the Device Status Register.

Table 246. Set Device Status command bit description

Bit	Symbol	Value	Description	Reset value
0	CON		The Connect bit indicates the current connect status of the device. It controls the CONNECT output pin, used for SoftConnect. Reading the connect bit returns the current connect status. This bit is cleared by hardware when the V _{BUS} status input is LOW for more than 3 ms. The 3 ms delay filters out temporary dips in the V _{BUS} voltage.	0
		0	Writing a 0 will make the CONNECT pin go HIGH.	
		1	Writing a 1 will make the CONNECT pin go LOW.	
1	CON_CH		Connect Change.	0
		0	This bit is cleared when read.	
		1	This bit is set when the device's pull-up resistor is disconnected because V _{BUS} disappeared. The DEV_STAT interrupt is generated when this bit is 1.	
2	SUS		Suspend: The Suspend bit represents the current suspend state. When the device is suspended (SUS = 1) and the CPU writes a 0 into it, the device will generate a remote wake-up. This will only happen when the device is connected (CON = 1). When the device is not connected or not suspended, writing a 0 has no effect. Writing a 1 to this bit has no effect.	0
		0	This bit is reset to 0 on any activity.	
		1	This bit is set to 1 when the device hasn't seen any activity on its upstream port for more than 3 ms.	
3	SUS_CH		Suspend (SUS) bit change indicator. The SUS bit can toggle because: • The device goes into the suspended state. • The device is disconnected. • The device receives resume signalling on its upstream port. This bit is cleared when read.	0
		0	SUS bit not changed.	
		1	SUS bit changed. At the same time a DEV_STAT interrupt is generated.	

Table 246. Set Device Status command bit description

Bit	Symbol	Value	Description	Reset value
4	RST		Bus Reset bit. On a bus reset, the device will automatically go to the default state. In the default state: • Device is unconfigured. • Will respond to address 0. • Control endpoint will be in the Stalled state. • All endpoints are unrealized except control endpoints EP0 and EP1. • Data toggling is reset for all endpoints. • All buffers are cleared. • There is no change to the endpoint interrupt status. • DEV_STAT interrupt is generated. Note: Bus resets are ignored when the device is not connected (CON=0).	0
		0	This bit is cleared when read.	
		1	This bit is set when the device receives a bus reset. A DEV_STAT interrupt is generated.	
7:5	-		Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

11.12.7 Get Device Status (Command: 0xFE, Data: read 1 byte)

The Get Device Status command returns the Device Status Register. Reading the device status returns 1 byte of data. The bit field definition is same as the Set Device Status Register as shown in [Table 246](#).

Remark: To ensure correct operation, the DEV_STAT bit of USBDevIntSt must be cleared before executing the Get Device Status command.

11.12.8 Get Error Code (Command: 0xFF, Data: read 1 byte)

Different error conditions can arise inside the SIE. The Get Error Code command returns the last error code that occurred. The 4 least significant bits form the error code.

Table 247. Get Error Code command bit description

Bit	Symbol	Value	Description	Reset value
3:0	EC		Error Code.	0x0
		0000	No Error.	
		0001	PID Encoding Error.	
		0010	Unknown PID.	
		0011	Unexpected Packet - any packet sequence violation from the specification.	
		0100	Error in Token CRC.	
		0101	Error in Data CRC.	
		0110	Time Out Error.	
		0111	Babble.	
		1000	Error in End of Packet.	
		1001	Sent/Received NAK.	
		1010	Sent Stall.	
		1011	Buffer Overrun Error.	
		1100	Sent Empty Packet (ISO Endpoints only).	
		1101	Bitstuff Error.	
		1110	Error in Sync.	
		1111	Wrong Toggle Bit in Data PID, ignored data.	
4	EA	-	The Error Active bit will be reset once this register is read.	
7:5	-		Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

11.12.9 Read Error Status (Command: 0xFB, Data: read 1 byte)

This command reads the 8-bit Error register from the USB device. This register records which error events have recently occurred in the SIE. If any of these bits are set, the ERR_INT bit of USBDevIntSt is set. The error bits are cleared after reading this register.

Table 248. Read Error Status command bit description

Bit	Symbol	Description	Reset value
0	PID_ERR	PID encoding error or Unknown PID or Token CRC.	0
1	UEPKT	Unexpected Packet - any packet sequence violation from the specification.	0
2	DCRC	Data CRC error.	0
3	TIMEOUT	Time out error.	0
4	EOP	End of packet error.	0
5	B_OVRN	Buffer Overrun.	0
6	BTSTF	Bit stuff error.	0
7	TGL_ERR	Wrong toggle bit in data PID, ignored data.	0

11.12.10 Select Endpoint (Command: 0x00 - 0x1F, Data: read 1 byte (optional))

The Select Endpoint command initializes an internal pointer to the start of the selected buffer in EP_RAM. Optionally, this command can be followed by a data read, which returns some additional information on the packet(s) in the endpoint buffer(s). The command code of the Select Endpoint command is equal to the physical endpoint number. In the case of a single buffered endpoint the B_2_FULL bit is not valid.

Table 249. Select Endpoint command bit description

Bit	Symbol	Value	Description	Reset value
0	FE		Full/Empty. This bit indicates the full or empty status of the endpoint buffer(s). For IN endpoints, the FE bit gives the ANDed result of the B_1_FULL and B_2_FULL bits. For OUT endpoints, the FE bit gives ORed result of the B_1_FULL and B_2_FULL bits. For single buffered endpoints, this bit simply reflects the status of B_1_FULL.	0
		0	For an IN endpoint, at least one write endpoint buffer is empty.	
		1	For an OUT endpoint, at least one endpoint read buffer is full.	
1	ST		Stalled endpoint indicator.	0
		0	The selected endpoint is not stalled.	
		1	The selected endpoint is stalled.	
2	STP		SETUP bit: the value of this bit is updated after each successfully received packet (i.e. an ACKed package on that particular physical endpoint).	0
		0	The STP bit is cleared by doing a Select Endpoint/Clear Interrupt on this endpoint.	
		1	The last received packet for the selected endpoint was a SETUP packet.	
3	PO		Packet over-written bit.	0
		0	The PO bit is cleared by the 'Select Endpoint/Clear Interrupt' command.	
		1	The previously received packet was over-written by a SETUP packet.	
4	EPN		EP NAKed bit indicates sending of a NAK. If the host sends an OUT packet to a filled OUT buffer, the device returns NAK. If the host sends an IN token packet to an empty IN buffer, the device returns NAK.	0
		0	The EPN bit is reset after the device has sent an ACK after an OUT packet or when the device has seen an ACK after sending an IN packet.	
		1	The EPN bit is set when a NAK is sent and the interrupt on NAK feature is enabled.	
5	B_1_FULL		The buffer 1 status.	0
		0	Buffer 1 is empty.	
		1	Buffer 1 is full.	
6	B_2_FULL		The buffer 2 status.	0
		0	Buffer 2 is empty.	
		1	Buffer 2 is full.	
7	-	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

11.12.11 Select Endpoint/Clear Interrupt (Command: 0x40 - 0x5F, Data: read 1 byte)

Commands 0x40 to 0x5F are identical to their Select Endpoint equivalents, with the following differences:

- They clear the bit corresponding to the endpoint in the USBEpIntSt register.
- In case of a control OUT endpoint, they clear the STP and PO bits in the corresponding Select Endpoint Register.
- Reading one byte is obligatory.

Remark: This command may be invoked by using the USBCmdCode and USBCmdData registers, or by setting the corresponding bit in USBEpIntClr. For ease of use, using the USBEpIntClr register is recommended.

11.12.12 Set Endpoint Status (Command: 0x40 - 0x55, Data: write 1 byte (optional))

The Set Endpoint Status command sets status bits 7:5 and 0 of the endpoint. The Command Code of Set Endpoint Status is equal to the sum of 0x40 and the physical endpoint number in hex. Not all bits can be set for all types of endpoints.

Table 250. Set Endpoint Status command bit description

Bit	Symbol	Value	Description	Reset value
0	ST		Stalled endpoint bit. A Stalled control endpoint is automatically unstalled when it receives a SETUP token, regardless of the content of the packet. If the endpoint should stay in its stalled state, the CPU can stall it again by setting this bit. When a stalled endpoint is unstalled - either by the Set Endpoint Status command or by receiving a SETUP token - it is also re-initialized. This flushes the buffer: in case of an OUT buffer it waits for a DATA 0 PID; in case of an IN buffer it writes a DATA 0 PID. There is no change of the interrupt status of the endpoint. When already unstalled, writing a zero to this bit initializes the endpoint. When an endpoint is stalled by the Set Endpoint Status command, it is also re-initialized.	0
		0	The endpoint is unstalled.	
		1	The endpoint is stalled.	
4:1	-	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA
5	DA		Disabled endpoint bit.	0
		0	The endpoint is enabled.	
		1	The endpoint is disabled.	
6	RF_MO		Rate Feedback Mode.	0
		0	Interrupt endpoint is in the Toggle mode.	
		1	Interrupt endpoint is in the Rate Feedback mode. This means that transfer takes place without data toggle bit.	
7	CND_ST		Conditional Stall bit.	0
		0	Unstalls both control endpoints.	
		1	Stall both control endpoints, unless the STP bit is set in the Select Endpoint register. It is defined only for control OUT endpoints.	

11.12.13 Clear Buffer (Command: 0xF2, Data: read 1 byte (optional))

When an OUT packet sent by the host has been received successfully, an internal hardware FIFO status Buffer_Full flag is set. All subsequent packets will be refused by returning a NAK. When the device software has read the data, it should free the buffer by issuing the Clear Buffer command. This clears the internal Buffer_Full flag. When the buffer is cleared, new packets will be accepted.

When bit 0 of the optional data byte is 1, the previously received packet was over-written by a SETUP packet. The Packet over-written bit is used only in control transfers. According to the USB specification, a SETUP packet should be accepted irrespective of the buffer status. The software should always check the status of the PO bit after reading

the SETUP data. If it is set then it should discard the previously read data, clear the PO bit by issuing a Select Endpoint/Clear Interrupt command, read the new SETUP data and again check the status of the PO bit.

See [Section 11.14 “Slave mode operation”](#) for a description of when this command is used.

Table 251. Clear Buffer command bit description

Bit	Symbol	Value	Description	Reset value
0	PO		Packet over-written bit. This bit is only applicable to the control endpoint EP0.	0
		0	The previously received packet is intact.	
		1	The previously received packet was over-written by a later SETUP packet.	
7:1	-	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

11.12.14 Validate Buffer (Command: 0xFA, Data: none)

When the CPU has written data into an IN buffer, software should issue a Validate Buffer command. This tells hardware that the buffer is ready for sending on the USB bus. Hardware will send the contents of the buffer when the next IN token packet is received.

Internally, there is a hardware FIFO status flag called Buffer_Full. This flag is set by the Validate Buffer command and cleared when the data has been sent on the USB bus and the buffer is empty.

A control IN buffer cannot be validated when its corresponding OUT buffer has the Packet Over-written (PO) bit (see the Clear Buffer Register) set or contains a pending SETUP packet. For the control endpoint the validated buffer will be invalidated when a SETUP packet is received.

See [Section 11.14 “Slave mode operation”](#) for a description of when this command is used.

11.13 USB device controller initialization

The LPC176x/5x USB device controller initialization includes the following steps:

1. Enable the device controller by setting the PCUSB bit of PCONP.
2. Configure and enable the PLL and Clock Dividers to provide 48 MHz for usbcclk and the desired frequency for cclk. For the procedure for determining the PLL setting and configuration, see [Section 4.5.11 “Procedure for determining PLL0 settings”](#).
3. Enable the device controller clocks by setting DEV_CLK_EN and AHB_CLK_EN bits in the USBCLKCtrl register. Poll the respective clock bits in the USBCLKSt register until they are set.
4. Enable the USB pin functions by writing to the corresponding PINSEL register.
5. Disable the pull-ups and pull-downs on the V_{BUS} pin using the corresponding PINMODE register by putting the pin in the “pin has neither pull-up nor pull-down resistor enabled” mode. See [Section 8.4 “Pin mode select register values”](#).
6. Set USBEPIn and USBMaxPSize registers for EP0 and EP1, and wait until the EP_RLZED bit in USBDevIntSt is set so that EP0 and EP1 are realized.

7. Enable endpoint interrupts (Slave mode):
 - Clear all endpoint interrupts using USBEpIntClr.
 - Clear any device interrupts using USBDevIntClr.
 - Enable Slave mode for the desired endpoints by setting the corresponding bits in USBEpIntEn.
 - Set the priority of each enabled interrupt using USBEpIntPri.
 - Configure the desired interrupt mode using the SIE Set Mode command.
 - Enable device interrupts using USBDevIntEn (normally DEV_STAT, EP_SLOW, and possibly EP_FAST).
8. Configure the DMA (DMA mode):
 - Disable DMA operation for all endpoints using USBEpDMADis.
 - Clear any pending DMA requests using USBDMARClr.
 - Clear all DMA interrupts using USBEoTIntClr, USBNDDRIntClr, and USBSysErrIntClr.
 - Prepare the UDCA in system memory.
 - Write the desired address for the UDCA to USBUDCAH.
 - Enable the desired endpoints for DMA operation using USBEpDMAEn.
 - Set EOT, DDR, and ERR bits in USBDMAIntEn.
9. Install USB interrupt handler in the NVIC by writing its address to the appropriate vector table location and enabling the USB interrupt in the NVIC.
10. Set default USB address to 0x0 and DEV_EN to 1 using the SIE Set Address command. A bus reset will also cause this to happen.
11. Set CON bit to 1 to make CONNECT active using the SIE Set Device Status command.

The configuration of the endpoints varies depending on the software application. By default, all the endpoints are disabled except control endpoints EP0 and EP1. Additional endpoints are enabled and configured by software after a SET_CONFIGURATION or SET_INTERFACE device request is received from the host.

11.14 Slave mode operation

In Slave mode, the CPU transfers data between RAM and the endpoint buffer using the Register Interface.

11.14.1 Interrupt generation

In slave mode, data packet transfer between RAM and an endpoint buffer can be initiated in response to an endpoint interrupt. Endpoint interrupts are enabled using the USBEpIntEn register, and are observable in the USBEpIntSt register.

All non-isochronous OUT endpoints generate an endpoint interrupt when they receive a packet without an error. All non-isochronous IN endpoints generate an interrupt when a packet is successfully transmitted, or when a NAK handshake is sent on the bus and the interrupt on NAK feature is enabled.

For Isochronous endpoints, transfer of data is done when the FRAME interrupt (in USBDevIntSt) occurs.

11.14.2 Data transfer for OUT endpoints

When the software wants to read the data from an endpoint buffer it should set the RD_EN bit and program LOG_ENDPOINT with the desired endpoint number in the USBCtrl register. The control logic will fetch the packet length to the USBRxPLen register, and set the PKT_RDY bit ([Table 217](#)).

Software can now start reading the data from the USBRxData register ([Table 216](#)). When the end of packet is reached, the RD_EN bit is cleared, and the RxENDPKT bit is set in the USBDevSt register. Software now issues a Clear Buffer (refer to [Table 251](#)) command. The endpoint is now ready to accept the next packet. For OUT isochronous endpoints, the next packet will be received irrespective of whether the buffer has been cleared. Any data not read from the buffer before the end of the frame is lost. See [Section 11.16 “Double buffered endpoint operation”](#) for more details.

If the software clears RD_EN before the entire packet is read, reading is terminated, and the data remains in the endpoint's buffer. When RD_EN is set again for this endpoint, the data will be read from the beginning.

11.14.3 Data transfer for IN endpoints

When writing data to an endpoint buffer, WR_EN ([Section 11.10.5.5 “USB Control register \(USBCtrl - 0x5000 C228\)”](#)) is set and software writes to the number of bytes it is going to send in the packet to the USBTxPLen register ([Section 11.10.5.4](#)). It can then write data continuously in the USBTxData register.

When the number of bytes programmed in USBTxPLen have been written to USBTxData, the WR_EN bit is cleared, and the TxENDPKT bit is set in the USBDevIntSt register. Software issues a Validate Buffer ([Section 11.12.14 “Validate Buffer \(Command: 0xFA, Data: none\)”](#)) command. The endpoint is now ready to send the packet. For IN isochronous endpoints, the data in the buffer will be sent only if the buffer is validated before the next FRAME interrupt occurs; otherwise, an empty packet will be sent in the next frame. If the software clears WR_EN before the entire packet is written, writing will start again from the beginning the next time WR_EN is set for this endpoint.

Both RD_EN and WR_EN can be high at the same time for the same logical endpoint. Interleaved read and write operation is possible.

11.15 DMA operation

In DMA mode, the DMA transfers data between RAM and the endpoint buffer.

The following sections discuss DMA mode operation. Background information is given in sections [Section 11.15.2 “USB device communication area”](#) and [Section 11.15.3 “Triggering the DMA engine”](#). The fields of the DMA Descriptor are described in [Section 11.15.4 “The DMA descriptor”](#). The last three sections describe DMA operation: [Section 11.15.5 “Non-isochronous endpoint operation”](#), [Section 11.15.6 “Isochronous endpoint operation”](#), and [Section 11.15.7 “Auto Length Transfer Extraction \(ATLE\) mode operation”](#).

11.15.1 Transfer terminology

Within this section three types of transfers are mentioned:

1. USB transfers – transfer of data over the USB bus. The USB 2.0 specification refers to these simply as transfers. Within this section they are referred to as USB transfers to distinguish them from DMA transfers. A USB transfer is composed of transactions. Each transaction is composed of packets.
2. DMA transfers – the transfer of data between an endpoint buffer and system memory (RAM).
3. Packet transfers – in this section, a packet transfer refers to the transfer of a packet of data between an endpoint buffer and system memory (RAM). A DMA transfer is composed of one or more packet transfers.

11.15.2 USB device communication area

The CPU and DMA controller communicate through a common area of memory, called the USB Device Communication Area, or UDCA. The UDCA is a 32-word array of DMA Descriptor Pointers (DDPs), each of which corresponds to a physical endpoint. Each DDP points to the start address of a DMA Descriptor, if one is defined for the endpoint. DDPs for unrealized endpoints and endpoints disabled for DMA operation are ignored and can be set to a NULL (0x0) value.

The start address of the UDCA is stored in the USBUDCAH register. The UDCA can reside at any 128-byte boundary of RAM that is accessible to both the CPU and DMA controller.

[Figure 31](#) illustrates the UDCA and its relationship to the UDCA Head (USBUDCAH) register and DMA Descriptors.

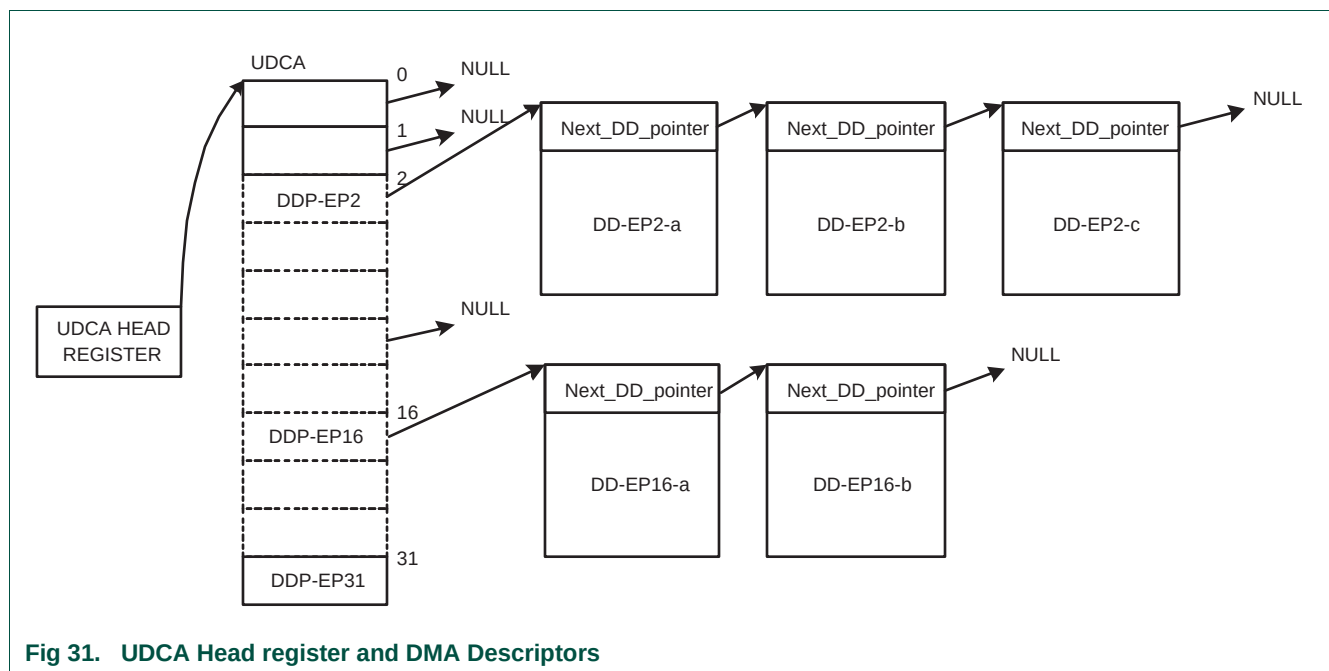


Fig 31. UDCA Head register and DMA Descriptors

11.15.3 Triggering the DMA engine

An endpoint raises a DMA request when Slave mode is disabled by setting the corresponding bit in the USBEPIntEn register to 0 ([Section 11.10.3.2](#)) and an endpoint interrupt occurs (see [Section 11.10.7.1 “USB DMA Request Status register \(USBDMARSt - 0x5000_C250\)”](#)).

A DMA transfer for an endpoint starts when the endpoint is enabled for DMA operation in USBEPDMARSt, the corresponding bit in USBDMARSt is set, and a valid DD is found for the endpoint.

All endpoints share a single DMA channel to minimize hardware overhead. If more than one DMA request is active in USBDMARSt, the endpoint with the lowest physical endpoint number is processed first.

In DMA mode, the bits corresponding to Interrupt on NAK for Bulk OUT and Interrupt OUT endpoints (INAK_BO and INAK_IO) should be set to 0 using the SIE Set Mode command ([Section 11.12.3](#)).

11.15.4 The DMA descriptor

DMA transfers are described by a data structure called the DMA Descriptor (DD).

DDs are placed in RAM. These descriptors can be located anywhere in on-chip RAM at word-aligned addresses.

DDs for non-isochronous endpoints are four words long. DDs for isochronous endpoints are five words long.

The parameters associated with a DMA transfer are:

- The start address of the DMA buffer
- The length of the DMA buffer
- The start address of the next DMA descriptor
- Control information
- Count information (number of bytes transferred)
- Status information

[Table 252](#) lists the DMA descriptor fields.

Table 252. DMA descriptor

Word position	Access (H/W)	Access (S/W)	Bit position	Description
0	R	R/W	31:0	Next_DD_pointer

Table 252. DMA descriptor

Word position	Access (H/W)	Access (S/W)	Bit position	Description
1	R	R/W	1:0	DMA_mode (00 -Normal; 01 - ATLE)
	R	R/W	2	Next_DD_valid (1 - valid; 0 - invalid)
	-	-	3	Reserved
	R	R/W	4	Isochronous_endpoint (1 - isochronous; 0 - non-isochronous)
	R	R/W	15:5	Max_packet_size
	R/W ^[1]	R/W	31:16	DMA_buffer_length This value is specified in bytes for non-isochronous endpoints and in number of packets for isochronous endpoints.
2	R/W	R/W	31:0	DMA_buffer_start_addr
3	R/W	R/I	0	DD_retired (To be initialized to 0)
	W	R/I	4:1	DD_status (To be initialized to 0000): 0000 - NotServiced 0001 - BeingServiced 0010 - NormalCompletion 0011 - DataUnderrun (short packet) 1000 - DataOverrun 1001 - SystemError
	W	R/I	5	Packet_valid (To be initialized to 0)
	W	R/I	6	LS_byte_extracted (ATLE mode) (To be initialized to 0)
	W	R/I	7	MS_byte_extracted (ATLE mode) (To be initialized to 0)
	R	W	13:8	Message_length_position (ATLE mode)
	-	-	15:14	Reserved
	R/W	R/I	31:16	Present_DMA_count (To be initialized to 0)
	R/W	R/W	31:0	Isochronous_packet_size_memory_address

[1] Write-only in ATLE mode

Legend: R - Read; W - Write; I - Initialize

11.15.4.1 Next_DD_pointer

Pointer to the memory location from where the next DMA descriptor will be fetched.

11.15.4.2 DMA_mode

Specifies the DMA mode of operation. Two modes have been defined: Normal and Automatic Transfer Length Extraction (ATLE) mode. In normal mode, software initializes the DMA_buffer_length for OUT endpoints. In ATLE mode, the DMA_buffer_length is extracted from the incoming data. See [Section 11.15.7 "Auto Length Transfer Extraction \(ATLE\) mode operation" on page 273](#) for more details.

11.15.4.3 Next_DD_valid

This bit indicates whether the software has prepared the next DMA descriptor. If set, the DMA engine fetches the new descriptor when it is finished with the current one.

11.15.4.4 Isochronous_endpoint

When set, this bit indicates that the descriptor belongs to an isochronous endpoint. Hence 5 words have to be read when fetching it.

11.15.4.5 Max_packet_size

The maximum packet size of the endpoint. This parameter is used while transferring the data for IN endpoints from the memory. It is used for OUT endpoints to detect the short packet. This is applicable to non-isochronous endpoints only. This field should be set to the same MPS value that is assigned for the endpoint using the USBMaxPSize register.

11.15.4.6 DMA_buffer_length

This indicates the depth of the DMA buffer allocated for transferring the data. The DMA engine will stop using this descriptor when this limit is reached and will look for the next descriptor.

In Normal mode operation, software sets this value for both IN and OUT endpoints. In ATLE mode operation, software sets this value for IN endpoints only. For OUT endpoints, hardware sets this value using the extracted length of the data stream.

For isochronous endpoints, DMA_buffer_length is specified in number of packets, for non-isochronous endpoints in bytes.

11.15.4.7 DMA_buffer_start_addr

The address where the data is read from or written to. This field is updated each time the DMA engine finishes transferring a packet.

11.15.4.8 DD_retired

This bit is set by hardware when the DMA engine finishes the current descriptor. This happens when the end of the buffer is reached, a short packet is transferred (non-isochronous endpoints), or an error condition is detected.

11.15.4.9 DD_status

The status of the DMA transfer is encoded in this field. The following codes are defined:

- **NotServiced** - No packet has been transferred yet.
- **BeingServiced** - At least one packet is transferred.
- **NormalCompletion** - The DD is retired because the end of the buffer is reached and there were no errors. The DD_retired bit is also set.
- **DataUnderrun** - Before reaching the end of the DMA buffer, the USB transfer is terminated because a short packet is received. The DD_retired bit is also set.
- **DataOverrun** - The end of the DMA buffer is reached in the middle of a packet transfer. This is an error situation. The DD_retired bit is set. The present DMA count field is equal to the value of DMA_buffer_length. The packet must be re-transmitted from the endpoint buffer in another DMA transfer. The corresponding EPxx_DMA_ENABLE bit in USBEpDMASt is cleared.
- **SystemError** - The DMA transfer being serviced is terminated because of an error on the AHB bus. The DD_retired bit is not set in this case. The corresponding EPxx_DMA_ENABLE in USBEpDMASt is cleared. Since a system error can happen while updating the DD, the DD fields in RAM may be unreliable.

11.15.4.10 Packet_valid

This bit is used for isochronous endpoints. It indicates whether the last packet transferred to the memory is received with errors or not. This bit is set if the packet is valid, i.e., it was received without errors. See [Section 11.15.6 “Isochronous endpoint operation” on page 271](#) for isochronous endpoint operation.

This bit is unnecessary for non-isochronous endpoints because a DMA request is generated only for packets without errors, and thus Packet_valid will always be set when the request is generated.

11.15.4.11 LS_byte_extracted

Used in ATLE mode. When set, this bit indicates that the Least Significant Byte (LSB) of the transfer length has been extracted. The extracted size is reflected in the DMA_buffer_length field, bits 23:16.

11.15.4.12 MS_byte_extracted

Used in ATLE mode. When set, this bit indicates that the Most Significant Byte (MSB) of the transfer size has been extracted. The size extracted is reflected in the DMA_buffer_length field, bits 31:24. Extraction stops when LS_Byte_extracted and MS_byte_extracted bits are set.

11.15.4.13 Present_DMA_count

The number of bytes transferred by the DMA engine. The DMA engine updates this field after completing each packet transfer.

For isochronous endpoints, Present_DMA_count is the number of packets transferred; for non-isochronous endpoints, Present_DMA_count is the number of bytes.

11.15.4.14 Message_length_position

Used in ATLE mode. This field gives the offset of the message length position embedded in the incoming data packets. This is applicable only for OUT endpoints. Offset 0 indicates that the message length starts from the first byte of the first packet.

11.15.4.15 Isochronous_packetsize_memory_address

The memory buffer address where the packet size information along with the frame number has to be transferred or fetched. See [Figure 32](#). This is applicable to isochronous endpoints only.

11.15.5 Non-isochronous endpoint operation

11.15.5.1 Setting up DMA transfers

Software prepares the DMA Descriptors (DDs) for those physical endpoints to be enabled for DMA transfer. These DDs are present in on-chip RAM. The start address of the first DD is programmed into the DMA Description pointer (DDP) location for the corresponding endpoint in the UDCA. Software then sets the EPxx_DMA_ENABLE bit for this endpoint in the USBEPDMAEn register ([Section 11.10.7.6](#)). The DMA_mode bit field in the descriptor is set to '00' for normal mode operation. All other DD fields are initialized as specified in [Table 252](#).

DMA operation is not supported for physical endpoints 0 and 1 (default control endpoints).

11.15.5.2 Finding DMA Descriptor

When there is a trigger for a DMA transfer for an endpoint, the DMA engine will first determine whether a new descriptor has to be fetched or not. A new descriptor does not have to be fetched if the last packet transferred was for the same endpoint and the DD is not yet in the retired state. An internal flag called DMA_PROCEED is used to identify this condition (see [Section 11.15.5.4 “Optimizing descriptor fetch” on page 270](#)).

If a new descriptor has to be read, the DMA engine will calculate the location of the DDP for this endpoint and will fetch the start address of the DD from this location. A DD start address at location zero is considered invalid. In this case the NDDR interrupt is raised. All other word-aligned addresses are considered valid.

When the DD is fetched, the DD status word (word 3) is read first and the status of the DD_retired bit is checked. If not set, DDP points to a valid DD. If DD_retired is set, the DMA engine will read the control word (word 1) of the DD.

If Next_DD_valid bit is set, the DMA engine will fetch the Next_DD_pointer field (word 0) of the DD and load it to the DDP. The new DDP is written to the UDCA area.

The full DD (4 words) will then be fetched from the address in the DDP. The DD will give the details of the DMA transfer to be done. The DMA engine will load its hardware resources with the information fetched from the DD (start address, DMA count etc.).

If Next_DD_valid is not set and DD_retired bit is set, the DMA engine raises the NDDR interrupt for this endpoint and clears the corresponding EPxx_DMA_ENABLE bit.

11.15.5.3 Transferring the data

For OUT endpoints, the current packet is read from the EP_RAM by the DMA Engine and transferred to on-chip RAM memory locations starting from DMA_buffer_start_addr. For IN endpoints, the data is fetched from on-chip RAM at DMA_buffer_start_addr and written to the EP_RAM. The DMA_buffer_start_addr and Present_DMA_count fields are updated after each packet is transferred.

11.15.5.4 Optimizing descriptor fetch

A DMA transfer normally involves multiple packet transfers. Hardware will not re-fetch a new DD from memory unless the endpoint changes. To indicate an ongoing multi-packet transfer, hardware sets an internal flag called DMA_PROCEED.

The DMA_PROCEED flag is cleared after the required number of bytes specified in the DMA_buffer_length field is transferred. It is also cleared when the software writes into the USBEPDMADis register. The ability to clear the DMA_PROCEED flag allows software to force the DD to be re-fetched for the next packet transfer. Writing all zeros into the USBEPDMADis register clears the DMA_PROCEED flag without disabling DMA operation for any endpoint.

11.15.5.5 Ending the packet transfer

On completing a packet transfer, the DMA engine writes back the DD with updated status information to the same memory location from where it was read. The DMA_buffer_start_addr, Present_DMA_count, and the DD_status fields in the DD are updated.

A DD can have the following types of completion:

Normal completion - If the current packet is fully transferred and the Present_DMA_count field equals the DMA_buffer_length, the DD has completed normally. The DD will be written back to memory with DD_retired set and DD_status set to NormalCompletion. The EOT interrupt is raised for this endpoint.

USB transfer end completion - If the current packet is fully transferred and its size is less than the Max_packet_size field, and the end of the DMA buffer is still not reached, the USB transfer end completion occurs. The DD will be written back to the memory with DD_retired set and DD_Status set to the DataUnderrun completion code. The EOT interrupt is raised for this endpoint.

Error completion - If the current packet is partially transferred i.e. the end of the DMA buffer is reached in the middle of the packet transfer, an error situation occurs. The DD is written back with DD_retired set and DD_status set to the DataOverrun status code. The EOT interrupt is raised for this endpoint and the corresponding bit in USBEpDMASt register is cleared. The packet will be re-sent from the endpoint buffer to memory when the corresponding EPxx_DMA_ENABLE bit is set again using the USBEpDMAEn register.

11.15.5.6 No_Packet DD

For an IN transfer, if the system does not have any data to send for a while, it can respond to an NDDR interrupt by programming a No_Packet DD. This is done by setting both the Max_packet_size and DMA_buffer_length fields in the DD to 0. On processing a No_Packet DD, the DMA engine clears the DMA request bit in USBDMARSt corresponding to the endpoint without transferring a packet. The DD is retired with a status code of NormalCompletion. This can be repeated as often as necessary. The device will respond to IN token packets on the USB bus with a NAK until a DD with a data packet is programmed and the DMA transfers the packet into the endpoint buffer.

11.15.6 Isochronous endpoint operation

For isochronous endpoints, the packet size can vary for each packet. There is one packet per isochronous endpoint for each frame.

11.15.6.1 Setting up DMA transfers

Software sets the isochronous endpoint bit to 1 in the DD, and programs the initial value of the Isochronous_packetsize_memory_address field. All other fields are initialized the same as for non-isochronous endpoints.

For isochronous endpoints, the DMA_buffer_length and Present_DMA_count fields are in frames rather than bytes.

11.15.6.2 Finding the DMA Descriptor

Finding the descriptors is done in the same way as that for a non-isochronous endpoint.

A DMA request will be placed for DMA-enabled isochronous endpoints on every FRAME interrupt. On processing the request, the DMA engine will fetch the descriptor and if Isochronous_endpoint is set, will fetch the Isochronous_packetsize_memory_address from the fifth word of the DD.

11.15.6.3 Transferring the Data

The data is transferred to or from the memory location `DMA_buffer_start_addr`. After the end of the packet transfer the `Present_DMA_count` value is incremented by 1.

The isochronous packet size is stored in memory as shown in [Figure 32](#). Each word in the packet size memory shown is divided into fields: `Frame_number` (bits 31 to 17), `Packet_valid` (bit 16), and `Packet_length` (bits 15 to 0). The space allocated for the packet size memory for a given DD should be `DMA_buffer_length` words in size – one word for each packet to transfer.

OUT endpoints

At the completion of each frame, the packet size is written to the address location in `Isochronous_packet_size_memory_address`, and `Isochronous_packet_size_memory_address` is incremented by 4.

IN endpoints

Only the `Packet_length` field of the isochronous packet size word is used. For each frame, an isochronous data packet of size specified by this field is transferred from the USB device to the host, and `Isochronous_packet_size_memory_address` is incremented by 4 at the end of the packet transfer. If `Packet_length` is zero, an empty packet will be sent by the USB device.

11.15.6.4 DMA descriptor completion

DDs for isochronous endpoints can only end with a status code of `NormalCompletion` since there is no short packet on Isochronous endpoints, and the USB transfer continues indefinitely until a `SystemError` occurs. There is no `DataOverrun` detection for isochronous endpoints.

11.15.6.5 Isochronous OUT Endpoint Operation Example

Assume that an isochronous endpoint is programmed for the transfer of 10 frames and that the transfer begins when the frame number is 21. After transferring four frames with packet sizes of 10, 15, 8 and 20 bytes without errors, the descriptor and memory map appear as shown in [Figure 32](#).

$\text{The_total_number_of_bytes_transferred} = 0x0A + 0x0F + 0x08 + 0x14 = 0x35$.

The `Packet_valid` bit (bit 16) of all the words in the packet length memory is set to 1.

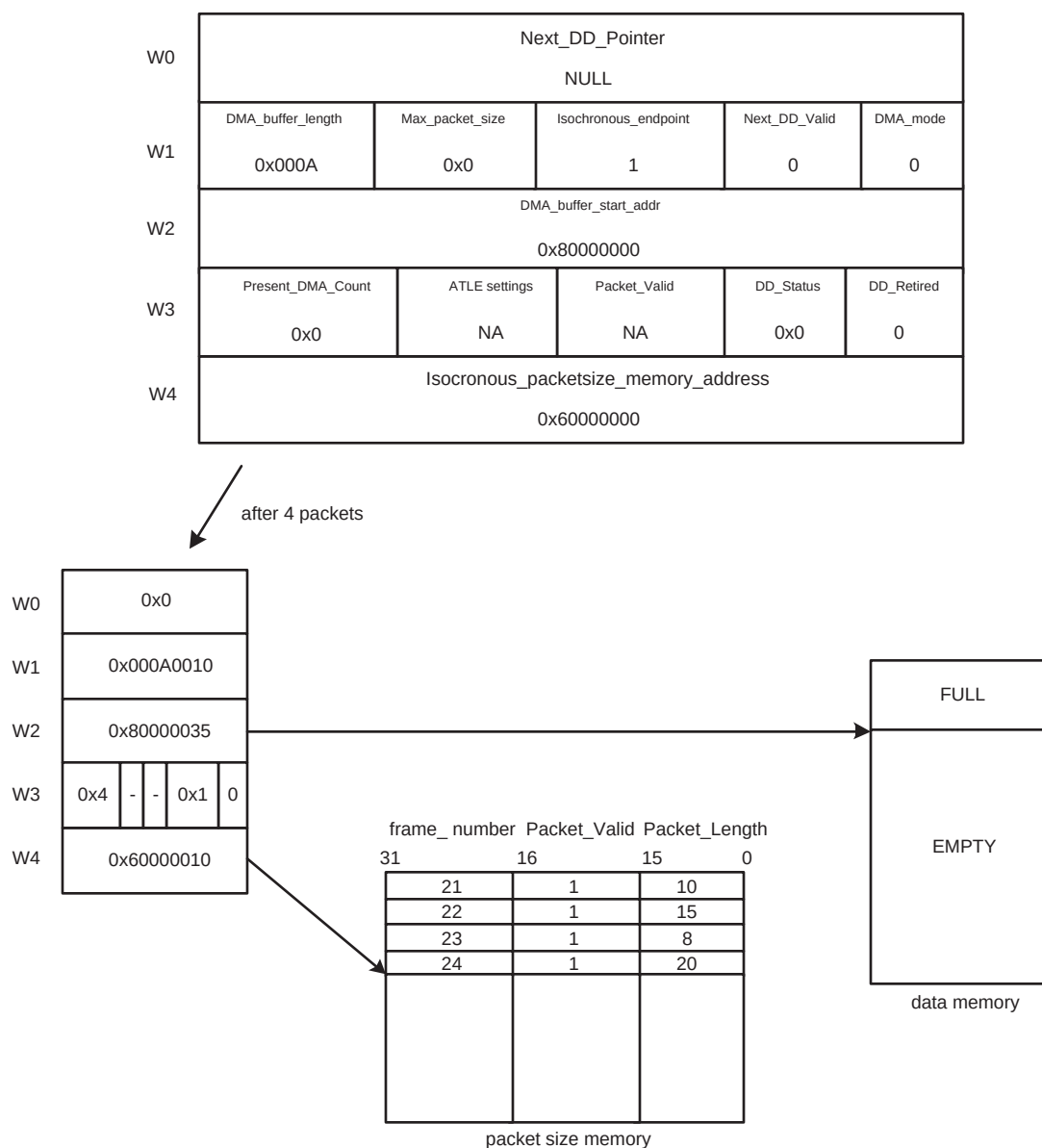


Fig 32. Isochronous OUT endpoint operation example

11.15.7 Auto Length Transfer Extraction (ATLE) mode operation

Some host drivers such as NDIS (Network Driver Interface Specification) host drivers are capable of concatenating small USB transfers (delta transfers) to form a single large USB transfer. For OUT USB transfers, the device hardware has to break up this concatenated transfer back into the original delta transfers and transfer them to separate DMA buffers. This is achieved by setting the DMA mode to Auto Transfer Length Extraction (ATLE) mode in the DMA descriptor. ATLE mode is supported for Bulk endpoints only.

OUT transfers in ATLE mode

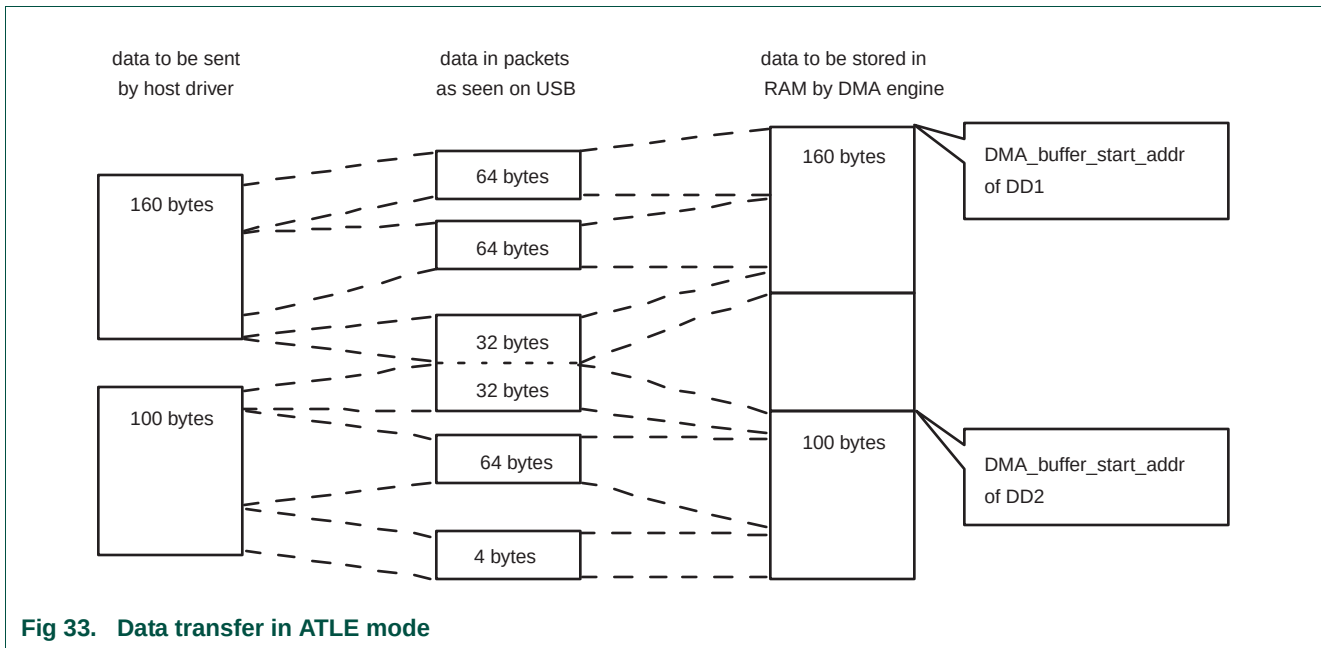


Fig 33. Data transfer in ATLE mode

Figure 33 shows a typical OUT USB transfer in ATLE mode, where the host concatenates two USB transfers of 160 bytes and 100 bytes, respectively. Given a MaxPacketSize of 64, the device hardware interprets this USB transfer as four packets of 64 bytes and a short packet of 4 bytes. The third and fourth packets are concatenated. Note that in Normal mode, the USB transfer would be interpreted as packets of 64, 64, 32, and 64 and 36 bytes.

It is now the responsibility of the DMA engine to separate these two USB transfers and put them in the memory locations in the DMA_buffer_start_addr field of DMA Descriptor 1 (DD1) and DMA Descriptor 2 (DD2).

Hardware reads the two-byte-wide DMA_buffer_length at the offset (from the start of the USB transfer) specified by Message_length_position from the incoming data packets and writes it in the DMA_buffer_length field of the DD. To ensure that both bytes of the DMA_buffer_length are extracted in the event they are split between two packets, the flags LS_byte_extracted and MS_byte_extracted are set by hardware after the respective byte is extracted. After the extraction of the MS byte, the DMA transfer continues as in the normal mode.

The flags LS_byte_extracted and MS_byte_extracted are set to 0 by software when preparing a new DD. Therefore, once a DD is retired, the transfer length is extracted again for the next DD.

If DD1 is retired during the transfer of a concatenated packet (such as the third packet in Figure 33), and DD2 is not programmed (Next_DD_valid field of DD1 is 0), then DD1 is retired with DD_status set to the DataOverrun status code. This is treated as an error condition and the corresponding EPxx_DMA_ENABLE bit of USBEPDMAst is cleared by hardware.

In ATLE mode, the last buffer length to be transferred always ends with a short or empty packet indicating the end of the USB transfer. If the concatenated transfer lengths are such that the USB transfer ends on a MaxPacketSize packet boundary, the (NDIS) host will send an empty packet to mark the end of the USB transfer.

IN transfers in ATLE mode

For IN USB transfers from the device to the host, DMA_buffer_length is set by the device software as in normal mode.

In ATLE mode, the device concatenates data from multiple DDs to form a single USB transfer. If a DD is retired in the middle of a packet (packet size is less than MaxPacketSize), the next DD referenced by Next_DD_pointer is fetched, and the remaining bytes to form a packet of MaxPacketSize are transferred from the next DD's buffer.

If the next DD is not programmed (i.e. Next_DD_valid field in DD is 0), and the DMA buffer length for the current DD has completed before the MaxPacketSize packet boundary, then the available bytes from current DD are sent as a short packet on USB, which marks the end of the USB transfer for the host.

If the last buffer length completes on a MaxPacketSize packet boundary, the device software must program the next DD with DMA_buffer_length field 0, so that an empty packet is sent by the device to mark the end of the USB transfer for the host.

11.15.7.1 Setting up the DMA transfer

For OUT endpoints, the host hardware needs to set the field Message_length_position in the DD. This indicates the start location of the message length in the incoming data packets. Also the device software has to set the DMA_buffer_length field to 0 for OUT endpoints because this field is updated by the device hardware after the extraction of the buffer length.

For IN endpoints, descriptors are set in the same way as in normal mode operation.

Since a single packet can be split between two DDs, software should always keep two DDs ready, except for the last DMA transfer which ends with a short or empty packet.

11.15.7.2 Finding the DMA Descriptor

DMA descriptors are found in the same way as the normal mode operation.

11.15.7.3 Transferring the Data

OUT endpoints

If the LS_byte_extracted or MS_byte_extracted bit in the status field is not set, the hardware will extract the transfer length from the data stream and program DMA_buffer_length. Once the extraction is complete both the LS_byte_extracted and MS_byte_extracted bits will be set.

IN endpoints

The DMA transfer proceeds as in normal mode and continues until the number of bytes transferred equals the DMA_buffer_length.

11.15.7.4 Ending the packet transfer

The DMA engine proceeds with the transfer until the number of bytes specified in the field `DMA_buffer_length` is transferred to or from on-chip RAM. Then the EOT interrupt will be generated. If this happens in the middle of the packet, the linked DD will get loaded and the remaining part of the packet gets transferred to or from the address pointed by the new DD.

OUT endpoints

If the linked DD is not valid and the packet is partially transferred to memory, the DD ends with `DataOverrun` status code set, and the DMA will be disabled for this endpoint. Otherwise `DD_status` will be updated with the `NormalCompletion` status code.

IN endpoints

If the linked DD is not valid and the packet is partially transferred to USB, the DD ends with a status code of `NormalCompletion` in the `DD_status` field. This situation corresponds to the end of the USB transfer, and the packet will be sent as a short packet. Also, when the linked DD is valid and buffer length is 0, an empty packet will be sent to indicate the end of the USB transfer.

11.16 Double buffered endpoint operation

The Bulk and Isochronous endpoints of the USB Device Controller are double buffered to increase data throughput.

When a double-buffered endpoint is realized, enough space for both endpoint buffers is automatically allocated in the EP_RAM. See [Section 11.10.4.1](#).

For the following discussion, the endpoint buffer currently accessible to the CPU or DMA engine for reading or writing is said to be the active buffer.

11.16.1 Bulk endpoints

For Bulk endpoints, the active endpoint buffer is switched by the SIE Clear Buffer or Validate Buffer commands.

The following example illustrates how double buffering works for a Bulk OUT endpoint in Slave mode:

Assume that both buffer 1 (B_1) and buffer 2 (B_2) are empty, and that the active buffer is B_1.

1. The host sends a data packet to the endpoint. The device hardware puts the packet into B_1, and generates an endpoint interrupt.
2. Software clears the endpoint interrupt and begins reading the packet data from B_1. While B_1 is still being read, the host sends a second packet, which device hardware places in B_2, and generates an endpoint interrupt.
3. Software is still reading from B_1 when the host attempts to send a third packet. Since both B_1 and B_2 are full, the device hardware responds with a NAK.
4. Software finishes reading the first packet from B_1 and sends a SIE Clear Buffer command to free B_1 to receive another packet. B_2 becomes the active buffer.

5. Software sends the SIE Select Endpoint command to read the Select Endpoint Register and test the FE bit. Software finds that the active buffer (B_2) has data (FE=1). Software clears the endpoint interrupt and begins reading the contents of B_2.
6. The host re-sends the third packet which device hardware places in B_1. An endpoint interrupt is generated.
7. Software finishes reading the second packet from B_2 and sends a SIE Clear Buffer command to free B_2 to receive another packet. B_1 becomes the active buffer. Software waits for the next endpoint interrupt to occur (it already has been generated back in step 6).
8. Software responds to the endpoint interrupt by clearing it and begins reading the third packet from B_1.
9. Software finishes reading the third packet from B_1 and sends a SIE Clear Buffer command to free B_1 to receive another packet. B_2 becomes the active buffer.
10. Software tests the FE bit and finds that the active buffer (B_2) is empty (FE=0).
11. Both B_1 and B_2 are empty. Software waits for the next endpoint interrupt to occur. The active buffer is now B_2. The next data packet sent by the host will be placed in B_2.

The following example illustrates how double buffering works for a Bulk IN endpoint in Slave mode:

Assume that both buffer 1 (B_1) and buffer 2 (B_2) are empty and that the active buffer is B_1. The interrupt on NAK feature is enabled.

1. The host requests a data packet by sending an IN token packet. The device responds with a NAK and generates an endpoint interrupt.
2. Software clears the endpoint interrupt. The device has three packets to send. Software fills B_1 with the first packet and sends a SIE Validate Buffer command. The active buffer is switched to B_2.
3. Software sends the SIE Select Endpoint command to read the Select Endpoint Register and test the FE bit. It finds that B_2 is empty (FE=0) and fills B_2 with the second packet. Software sends a SIE Validate Buffer command, and the active buffer is switched to B_1.
4. Software waits for the endpoint interrupt to occur.
5. The device successfully sends the packet in B_1 and clears the buffer. An endpoint interrupt occurs.
6. Software clears the endpoint interrupt. Software fills B_1 with the third packet and validates it using the SIE Validate Buffer command. The active buffer is switched to B_2.
7. The device successfully sends the second packet from B_2 and generates an endpoint interrupt.
8. Software has no more packets to send, so it simply clears the interrupt.
9. The device successfully sends the third packet from B_1 and generates an endpoint interrupt.
10. Software has no more packets to send, so it simply clears the interrupt.

11. Both B_1 and B_2 are empty, and the active buffer is B_2. The next packet written by software will go into B_2.

In DMA mode, switching of the active buffer is handled automatically in hardware. For Bulk IN endpoints, proactively filling an endpoint buffer to take advantage of the double buffering can be accomplished by manually starting a packet transfer using the USBDMARSet register.

11.16.2 Isochronous endpoints

For isochronous endpoints, the active data buffer is switched by hardware when the FRAME interrupt occurs. The SIE Clear Buffer and Validate Buffer commands do not cause the active buffer to be switched.

Double buffering allows the software to make full use of the frame interval writing or reading a packet to or from the active buffer, while the packet in the other buffer is being sent or received on the bus.

For an OUT isochronous endpoint, any data not read from the active buffer before the end of the frame is lost when it switches.

For an IN isochronous endpoint, if the active buffer is not validated before the end of the frame, an empty packet is sent on the bus when the active buffer is switched, and its contents will be overwritten when it becomes active again.